

EXHIBIT BOOK

Criterion (E) — Original Scientific or Scholarly
Research Contributions to the Academic Field

8 C.F.R. § 204.5(i)(3)(i)(E)

Billing Event Reconstruction and
Payment-Delay Early-Warning for
Clinical Laboratory Settings

Beneficiary: Yajie Xu

Tool release: v1.4.0 (2026-09-22)

24 exhibits · sections E1-E4

Master Index of Exhibits

E1 — Method and Originality Position (Exhibits 1–4)

Exhibit 1	Method Report	4
Exhibit 2	Prior Work Comparison	11
Exhibit 3	Candidate Method Designs	16
Exhibit 4	Research Protocol and Change Record	19

E2 — The Tool Itself (Exhibits 5–16)

Exhibit 5	Tool Implementation — Linkage Rules	23
Exhibit 6	Tool Implementation — Amount Engine	27
Exhibit 7	Tool Implementation — Snapshot and Labeling	32
Exhibit 8	Tool Implementation — Models	37
Exhibit 9	Repository — Home	43
Exhibit 10	Repository — Commit History	47
Exhibit 11	Repository — Releases	52
Exhibit 12	Repository — Representative Code	54
Exhibit 13	Interface — Project and Data Status	58
Exhibit 14	Interface — Event Reconstruction	60
Exhibit 15	Interface — Manual Review	62
Exhibit 16	Interface — Delay Early-Warning	64

E3 — Personal Role (Exhibits 17–18)

Exhibit 17	Research Decision Records	67
Exhibit 18	Version History	76

E4 — Verification (Exhibits 19–24)

Exhibit 19	Validation Against Held-Out Adjudicated Labels	80
Exhibit 20	Acceptance Test Report	83
Exhibit 21	Data Card	85
Exhibit 22	Data Sample — Bills	88
Exhibit 23	Data Sample — Events	99
Exhibit 24	Data Sample — Allocations	109

E1 — Method and Originality Position (Exhibits 1–4)

Exhibit 1

Method Report

Method report v2.0: research questions, related work, data, event dictionary and prediction target, candidate methods, splits and leakage control, results, limitations.

Method Report — Billing Event Reconstruction and Payment-Delay Early-Warning for Clinical Laboratory Settings

- Version: 2.0
 - Date: 2026-09-22
-

1. Research Questions

- RQ1: Under multi-source, missing or conflicting records, does uncertainty-preserving event reconstruction (explicit lineages, conflicts, needs-review queue) reduce wrong links and control manual review volume, compared with simple identifier joins and appropriate existing methods?
- RQ2: Does incorporating available record-quality and processing-stage signals improve within-horizon prediction of first-observable positive payment allocation vs. no-payment closure vs. still-open, relative to aging-rule and basic baselines?

2. Related Work and Position

- Fellegi & Sunter (1969): decision-theoretic record linkage with a possible-link disposition; the needs-review tier operationalizes this disposition.
- Christen (2012) and van der Aalst (2016): general matching and event-log analysis; this work adds billing-domain event semantics and as-of point-in-time discipline.
- Deep Claim (arXiv:2007.06229): predicts payer responses pre-submission from claims; this work differs in scope: post-submission billing-event reconstruction plus an observation-window-aware competing-event forecast.
- Fine & Gray (1999) / Kaplan & Meier (1958): competing-risk and censoring foundations; a discrete-time interval formulation is adopted for irregular observation.

- Zeng et al. (2007), Schoonbee (2022): invoice-lateness prediction; the aging baseline is informed by these, with explicit censoring treatment added.

Reference list: research/prior_work.csv (nine references). Reading scope is recorded per reference: bibliography-verified (metadata confirmed against the publisher's record) or bib-and-abstract-verified (metadata plus abstract read in full). Full-text analysis is not claimed for any reference at this stage.

3. Data and Permissions

- The study uses de-identified billing event records from three participating clinical laboratory organizations, referred to by site tokens CPL, TRI and NDX under the data-use arrangement. Site tokens are used throughout; organizational identities are recorded in the study documentation.
- Data window: 2022-07-01 to 2023-11-15. Records carry per-site local timezones (including daylight-saving transitions); submission and payment activity follows weekday-dominant rhythms with a mix of same-day and lagged record entry.
- Data quality issues observed and handled: duplicate imports, cross-organization duplicate bill numbers, missing correction references, timezone-less timestamps, invalid amounts and non-USD currencies. These are isolated by the schema validator, never silently repaired, and reported in the run documentation.
- Manually adjudicated reference labels (record pair lineage membership and observed outcome times) are held out from all model inputs and used only for evaluation.
- Real-mode processing of identified data remains disabled; see governance/data_permissions.json.

4. Event Dictionary and Prediction Target

- Event types: SUBMITTED, CORRECTED, PAYMENT_POSTED, PAYMENT_REVERSED, VOID_VERSION,

CLOSED_NO_PAYMENT, REOPENED, ADJUSTMENT, STATUS_UPDATED, UNKNOWN (with review).

- Target: within $H=30$ days after landmark t (first observable submission + 7 days), predict the first observable positive payment allocation, first no-payment closure, or still-open. Not final bank cash, not full settlement.
- Protocol v1.1 (PC01, 2024-08-06): interval granularity 1 -> 5 days after sparsity findings at 1-day resolution.

5. Candidate Methods

- Module A: A0 exact-id baseline (native previous_record_id with org/cycle/time checks); A1 constrained linkage (R2 same-bill-id version chains; R3 amount/date-window weak evidence with uniqueness gating). Accepted / rejected / needs-review tiers; no unconstrained transitive chaining; manual decisions applied as a separate layer.
- Module B: B0 age/stage empirical-risk baseline (abstains on unsupported cells); B1 discrete-time multinomial logistic regression; C1 B1 plus reconstruction-quality features. Class probabilities per interval with the product-rule cumulative curves; sum-to-one enforced (T16).

6. Splits and Leakage Control

- Temporal split grouped by lineage, 60/20/20; test portion locked.
- Features computed strictly from records observable at t (T10); reference labels and outcome fields excluded from features (T13); lineages never span splits (T14); preprocessing fitted on train only (T15).

7. Results (locked test set)

Cohort flow (all records of the reference run):

bills 1,164 -> reconstructed lineages 1,044 -> snapshots 959

-> locked test snapshots 192. Snapshots whose outcome was already

determined before the landmark are not prediction targets and are excluded at snapshot construction.

Module A (held-out adjudicated adjacency evaluation):

- reference pairs 95; predicted 123
- true positives 94; false positives 29; false negatives 1
- precision 0.7642 (95% Wilson interval 0.680-0.832); recall 0.9895 (0.943-0.999); refusal ratio 0.0956
- AO unconstrained baseline (same-org chaining in submission order, no evidence requirements): precision 0.0017, recall 0.0211. The constrained method's precision is materially higher (0.7642 vs 0.0017); the AO baseline is retained only as a deliberately weak lower bound and is not presented as a competing method.
- The weak-evidence rule R3 is the dominant false-positive source; multi-candidate ambiguity is routed to review instead of being forced.

Module B (snapshot-level, n=192 test snapshots; outcome mix

{"payment": 576, "still_open": 375, "close_no_payment": 8} over all

959 snapshots):

- B0: snap log-loss 0.7149, Brier 0.4943
- B1: snap log-loss 0.7123, Brier 0.4908
(absolute delta -0.0026; relative improvement 0.36%)
- C1: snap log-loss 0.7124, Brier 0.4915
(delta vs B1 +0.0001 — no incremental value)
- The B1-vs-B0 delta of 0.36% does not clear any practically meaningful improvement threshold; it is reported as a neutral result. No pre-specified gain margin was frozen in the protocol; that omission is

recorded as a study-design limitation (section 9).

8. Negative / Neutral Findings (reported as-is)

- C1 quality features add no measurable value on the locked test set.
- An early implementation with balanced class weighting distorted conditional probabilities approximately 4x; corrected after diagnostics (2024-12).
- Close-no-payment events are rare in the observation window (n=8); the model abstains rather than fabricating probabilities where support is insufficient.
- The candidate B1 improves snapshot log-loss over the B0 baseline by only 0.36% on the locked test set — not a substantive gain.

9. Applicability and Limitations

- Applicable condition: multi-source billing exports with availability-time semantics, USD-denominated amounts, explicit or inferable version chains.
- Not applicable: final collection forecasting, cross-currency aggregation, identified patient-level analytics without authorization.
- Limitations: the participating sites span three ownership models in one national context; generalization to other settings requires further validation. Landmark and horizon are fixed study defaults pending operational confirmation. No external independent validation has been completed to date. No pre-specified gain margin for the Module B comparison was frozen in the research protocol; the post-hoc interpretation of the observed 0.36% delta is therefore descriptive, not confirmatory. Uncertainty intervals for Module B snapshot-level differences are not reported because the snapshot estimates are not independent.

10. Contributions

- The research design, decision records (research/decisions/, D01-D06 and PC01) and

the analysis were carried out by the applicant; engineering execution used computational tooling under her direction. Individual decisions trace to code and tests as documented in the decision records.

11. Reproducibility

- Environment: environment.json (Python 3.12.13; historical snapshot 3.11.5).
- Commands: the standard pipeline entry (full pipeline); pytest (32 acceptance and regression tests); the validation entry (held-out evaluation). Full command references in README.
- Code and data access: <https://github.com/fanshiyu35/lab-billing-research>
- Run referenced: the v1.4 reference run (artifacts archived with the code release).

References

See research/prior_work.csv for the verified reference list.

Exhibit 2

Prior Work Comparison

Nine verified references across record linkage, claims prediction and competing-risk literature, with overlap and difference analysis.

E1-02 Prior Work

source: research/prior_work.csv — v1.4.0 (2026-09-22)

(12 fields; rendered as field dictionaries to preserve all columns)

Row 1

source_id	PW01
title	A Theory for Record Linkage
authors	Fellegi I.P.; Sunter A.B.
year	1969
url_doi	https://doi.org/10.1080/01621459.1969.10501049
reading_scope	bib-and-abstract-verified
problem_addressed	Optimal decision rule for linking records across files
data_method	Decision-theoretic model of comparison vectors with link/non-link/possible-link dispositions
limitations	Assumes comparison-pair errors follow known m/u probabilities; needs calibration
overlap_with_candidates	Theoretical basis for constrained linkage decision rule and needs-review class
proposed_difference	Explicit uncertainty-preserving tier (needs_review) with downstream conflict checks instead of forced dispositions
verifiable_location	JASA 64(328):1183-1210

Row 2

source_id	PW02
title	Data Matching: Concepts and Techniques for Record Linkage Entity Resolution and Duplicate Detection
authors	Christen P.
year	2012
url_doi	https://doi.org/10.1007/978-3-642-31164-2
reading_scope	bibliography-verified
problem_addressed	Broad survey of modern record linkage blocking similarity and evaluation
data_method	Survey of matching techniques blocking methods and evaluation measures
limitations	Generic survey; no billing-domain event semantics
overlap_with_candidates	Background for candidate blocking and precision/recall evaluation
proposed_difference	Domain-specific event dictionary and lineage semantics on top of generic matching
verifiable_location	Springer 2012

Row 3

source_id	PW03
title	Process Mining: Data Science in Action

authors	van der Aalst W.
year	2016
url_doi	https://doi.org/10.1007/978-3-662-49851-4
reading_scope	bibliography-verified
problem_addressed	Event log analysis conformance and discovery
data_method	Process mining over event logs with timestamps
limitations	Assumes process logs; billing events are messier multi-source
overlap_with_candidates	Framework for event normalization and reconstruction audit
proposed_difference	Explicit handling of late-arriving records and as-of truncation for point-in-time correctness
verifiable_location	Springer 2016 (2nd ed.)

Row 4

source_id	PW04
title	A Proportional Hazards Model for the Subdistribution of a Competing Risk
authors	Fine J.P.; Gray R.J.
year	1999
url_doi	https://doi.org/10.1080/01621459.1999.10474144
reading_scope	bib-and-abstract-verified
problem_addressed	Regression on cumulative incidence under competing risks
data_method	Semiparametric subdistribution hazard model
limitations	Continuous-time hazard formulation
overlap_with_candidates	Statistical grounding for competing payment/closure events
proposed_difference	Discrete-time interval-based conditional probabilities suitable for irregular observation
verifiable_location	JASA 94(446):496-509

Row 5

source_id	PW05
title	Nonparametric Estimation from Incomplete Observations
authors	Kaplan E.L.; Meier P.
year	1958
url_doi	https://doi.org/10.1080/01621459.1958.10501452
reading_scope	bibliography-verified
problem_addressed	Survival estimation with right censoring
data_method	Product-limit estimator
limitations	Classical; no covariates
overlap_with_candidates	Censoring conventions adopted in v1
proposed_difference	H-dependent interval decomposition with explicit open-state probability
verifiable_location	JASA 53(282):457-481

Row 6

source_id	PW06
title	Deep Claim: Payer Response Prediction from Claims Data with Deep Learning
authors	Kim B.-H.; Sridharan S.; Atwal A.; Ganapathi V.
year	2020
url_doi	https://arxiv.org/abs/2007.06229
reading_scope	bib-and-abstract-verified
problem_addressed	Denial and response-date prediction from claims
data_method	Deep multi-task architecture over 2.9M claims from two US health systems
limitations	Pre-submission features; no billing-event reconstruction; deep model not needed for this study
overlap_with_candidates	Closest applied work on payer response prediction
proposed_difference	Adds explicit reconstruction-quality features and uncertainty-preserving linkage as inputs; simpler transparent baseline comparison
verifiable_location	arXiv:2007.06229 (ICML 2020 HSYS Workshop)

Row 7

source_id	PW07
title	Predicting Invoice Payment Outcomes for Optimized Collections
authors	Zeng S.; Melville P.; Lang C.A.; et al.
year	2007
url_doi	https://prem-melville.com/publications/equitant-kdd07.pdf
reading_scope	bib-and-abstract-verified
problem_addressed	Classify invoice payment outcome into delay buckets for collection prioritization
data_method	Supervised learning over invoice-level features
limitations	Generic B2B invoices; no claim adjudication semantics
overlap_with_candidates	Baseline inspiration for delay-bucket prediction and aging features
proposed_difference	Event-level observation-window aware labeling instead of static outcome buckets
verifiable_location	KDD 2007 Workshop on Data Mining for Business Applications

Row 8

source_id	PW08
title	A machine-learning approach towards solving the invoice payment prediction problem
authors	Schoonbee S.
year	2022
url_doi	https://scielo.org.za/scielo.php?pid=S2224-78902022000400010&script=sci_arttext
reading_scope	bibliography-verified
problem_addressed	Interval-coded late-payment prediction for AR prioritization

data_method	ML classifier over invoice features into lateness intervals
limitations	South African industrial invoices; no censoring treatment
overlap_with_candidates	Supports aging-stage baseline design
proposed_difference	Censoring-aware competing-event labeling and refusal mechanism
verifiable_location	South African Journal of Industrial Engineering 33(4)

Row 9

source_id	PW09
title	Waste in the US Health Care System: Estimated Costs and Potential for Savings
authors	Shrank W.H.; Rogstad T.L.; Parekh N.
year	2019
url_doi	https://doi.org/10.1001/jama.2019.13978
reading_scope	bibliography-verified
problem_addressed	Administrative complexity as dominant waste source
data_method	Cost estimation study
limitations	Not a methods paper
overlap_with_candidates	Motivation for revenue-cycle analytics
proposed_difference	Not applicable
verifiable_location	JAMA 322(15):1501-1509

Exhibit 3

Candidate Method Designs

Candidate methods with pseudocode-level specification, inputs/outputs, failure conditions and evaluation plan.

E1-03 Candidate Methods

source: research/candidate_methods.md — v1.4.0 (2026-09-22)

Candidate methods (P01, protocol v1.0)

Research questions

- RQ1: Under multi-source, missing or conflicting records, does uncertainty-preserving event reconstruction (explicit lineages, conflicts, needs-review queue) reduce wrong links and control manual review volume, compared with simple identifier joins and appropriate existing methods?
- RQ2: Does incorporating available record-quality and processing-stage signals improve within-horizon prediction of first-observable positive payment allocation vs. no-payment closure vs. still-open, relative to aging-rule and basic baselines?

Both are hypotheses to be tested, not established improvements.

Module A candidates (reconstruction)

A0 - exact_id baseline

- Inputs: native previous_record_id plus confirmed mappings only.
- Decision: link when explicit reference exists and passes org/currency/time sanity checks.
- Failure condition: broken or circular references surface as conflicts.
- Cost: trivial. Source: standard deterministic join.

A1 - constrained_linkage candidate

- Candidate generation from business references, date windows and amount patterns within org + source system; never across orgs; never on payer/service token alone.
- Rules carry ID, input dependencies, action, conflict priority and provenance.
- Multi-candidate ambiguity is preserved as needs_review instead of forced picks.
- Confidence scores labeled UNVALIDATED unless calibrated.
- Difference vs A0: higher coverage with an explicit uncertainty tier and downstream conflict/amount consistency checks.

Module B candidates (prediction)

B0 - age/stage rule baseline

- Transparent rules or training-set empirical risk by aging bucket and known stage.
- Rules are not probabilities; empirical risk states its sample support.

B1 - discrete multinomial baseline

- Discrete-time competing-event model: per-day interval conditional probabilities (payment / close-no-payment / none) from t-time features only.
- $S(0)=1$; $F_payment(h)=\sum_{k \leq h} S(k-1) p_payment(k)$; F_close analogously; $S(h)=S(h-1) p_none(h)$; class probabilities sum to 1 at every step.

C1 - quality-augmented candidate

- Same modeling frame as B1 plus reconstruction-quality / missingness / conflict features computed strictly from data available at t.
- Explicitly a candidate to be tested; not claimed original in advance.

C2 (optional)

- Only if an independent reason appears (e.g. demonstrated non-linearity B1 cannot capture). Not added for appearance.

Evaluation

- Module A: precision/recall on adjudicable samples, wrong-merge/split counts, refusal ratio, coverage; unlabeled and unknown reported separately.
- Module B: log-loss, three-class Brier score, grouped calibration, class counts, refusal/censoring proportions; H-horizon metrics restricted to known-outcome or fully-covered cases with the selection disclosed.
- Ablations: drop quality features (C1 vs B1); use baseline reconstruction feeding B1.

Exhibit 4

Research Protocol and Change Record

Frozen research protocol (target definition, splits, metrics, conclusion conditions) and the versioned protocol change record.

Research protocol (frozen) and change record

source: research/protocol_v1_research_edition.json + protocol_changes.csv — v1.4.0

```
{
  "protocol_version": "1.1",
  "frozen_at": "2023-10-20",
  "research_questions": {
    "RQ1": "Does uncertainty-preserving event reconstruction reduce wrong links and control manual review volume vs simple identifier joins and appropriate existing methods?",
    "RQ2": "Does adding record-quality and processing-stage signals improve within-horizon first-event prediction vs aging and basic baselines?"
  },
  "target_definition": {
    "name": "FIRST_OBSERVED_POSITIVE_ALLOCATION_OR_NO_PAYMENT_CLOSURE",
    "status": "OPERATIONAL_CONFIRMATION_PENDING",
    "landmark_days": 7,
    "horizon_days": 30,
    "interval_days": 5,
    "outcomes": [
      "P_payment_by_H",
      "P_close_no_payment_by_H",
      "P_still_open_by_H"
    ],
    "censoring": "STOP_AT_RELIABLE_OBSERVATION_END",
    "insufficient_data_action": "ABSTAIN"
  },
  "linkage_methods": [
    "exact_id",
    "constrained_linkage"
  ],
  "models": [
    "B0_age_stage_baseline",
    "B1_discrete_multinomial",
    "C1_quality_augmented"
  ],
  "split": {
    "policy": "TEMPORAL_GROUPED_BY_LINEAGE",
    "lock_test_set": true,
    "ratios": [
      0.6,
      0.2,
      0.2
    ]
  },
  "metrics": {
    "module_a": [
      "precision",
      "recall",
      "wrong_merge_count",
      "wrong_split_count",
      "refusal_ratio",
      "coverage"
    ],
    "module_b": [
```

```

        "log_loss",
        "brier_three_class",
        "grouped_calibration",
        "class_counts",
        "refusal_ratio",
        "censoring_ratio"
    ]
},
"conclusion_conditions": {
    "positive": "Candidate improves a pre-specified metric on the locked test set beyond a pre-specified margin, with uncertainty intervals excluding null.",
    "negative": "No improvement, or worse. Reported as-is; not repaired by changing ground truth.",
    "inconclusive": "Insufficient sample or class counts; INSUFFICIENT_DATA recorded, no probabilities fabricated."
},
"feature_whitelist_frozen": true,
"updated_at": "2024-08-06",
"change_history": [
    {
        "change_id": "PC01",
        "date": "2024-08-06",
        "summary": "interval_days 1->5 after sparsity findings; see research/protocol_changes.csv"
    }
]
}

```

Change record

(8 fields; rendered as field dictionaries to preserve all columns)

Row 1

change_id	PC01
date	2024-08-06
affected_section	protocol section Module B interval
change	interval_days 1 -> 5 (H=30 becomes 6 intervals)
reason	Event sparsity at 1-day granularity made interval-level conditional probabilities numerically unstable (payment events concentrate around 10-30 days); 5-day intervals raise per-interval event rates to a learnable range while preserving within-horizon discrimination
evidence	Initial module B run 2024-08-05 (interval=1): B1 snap log-loss 11.3 vs B0 0.74; interval-level none share >95%
decision_by	Applicant
status	applied (protocol v1.1)

E2 — The Tool Itself (Exhibits 5–16)

Exhibit 5

Tool Implementation — Linkage Rules

Constrained linkage rules (R2 version chains, R3 weak-evidence candidates with uniqueness gating), with git blob reference.

Tool implementation — rules.py

src/lab_billing/reconstruct/rules.py — git blob e97482658b6e — v1.4.0 (2026-09-22)

```
"""Constrained linkage rules (module A, v0.7).

Each rule carries an ID, input dependencies, action, conflict priority and
source rationale. Rules only propose candidate pairs within an org and a
source system; they never cross orgs and never link on payer/service tokens
alone. When candidates are ambiguous the pair is queued for review instead
of being forced.

Rule inventory:
R1 native_previous_id      - native explicit predecessor (v0.2)
R2 same_bill_id_versions   - same org + bill_id, distinct version_ids
R3 amount_date_window      - weak evidence: same org, same payer/service,
                             close amounts, close dates (uniqueness required)
"""
from __future__ import annotations

from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Any

RULES = {
    "R1_native_previous_id": {
        "inputs": ["bills.previous_record_id"],
        "action": "accept link between referenced predecessor and successor",
        "conflict_priority": 1,
        "source": "native explicit reference in source data (spec section 10.2)",
    },
    "R2_same_bill_id_versions": {
        "inputs": ["bills.bill_id", "bills.version_id", "bills.org_token"],
        "action": "chain distinct versions of the same native bill number",
        "conflict_priority": 2,
        "source": "version chains within a native bill number (spec section 10.3)",
    },
    "R3_amount_date_window": {
        "inputs": ["bills.source_system", "bills.payer_token", "bills.service_token",
                  "bills.billed_amount_minor", "bills.submitted_at"],
        "action": "candidate pair on amount/date pattern; accept only when the "
                  "candidate is unique, otherwise queue for review",
        "conflict_priority": 3,
        "source": "business references, date windows and amount patterns "
                  "(spec section 10.3); weak evidence by design",
    },
}

@dataclass
class CandidatePair:
    from_id: str
    to_id: str
    rule_id: str
    evidence: str
    score: float = 0.0
```



```

score_kind: str = "UNVALIDATED_SCORE_UNLESS_CALIBRATED"

def generate_r2_candidates(bills: list[dict]) -> list[CandidatePair]:
    """Version chains within the same (org, bill_id)."""
    groups: dict[tuple, list[dict]] = {}
    for b in bills:
        groups.setdefault((b["org_token"], b["source_system"], b["bill_id"]),
[]) .append(b)
    out: list[CandidatePair] = []
    for (org, sysn, bid), members in groups.items():
        if len(members) < 2:
            continue
        # multiple rows claiming the same version => ambiguous
        versions: dict[str, list[dict]] = {}
        for m in members:
            versions.setdefault(m["version_id"], []).append(m)
        if any(len(v) > 1 for v in versions.values()):
            for m in members:
                out.append(CandidatePair(
                    from_id=m["record_id"], to_id="",
                    rule_id="R2_same_bill_id_versions",
                    evidence=f"ambiguous version set for {bid}",
                    score=0.0))
            continue
        ordered = sorted(members, key=lambda m: _ver_key(m["version_id"]))
        for prev, nxt in zip(ordered, ordered[1:]):
            out.append(CandidatePair(
                from_id=prev["record_id"], to_id=nxt["record_id"],
                rule_id="R2_same_bill_id_versions",
                evidence=f"version chain {prev['version_id']}->{nxt['version_id']} of
{bid}",
                score=0.9))
    return out

def generate_r3_candidates(bills: list[dict], window_days: int = 7,
                           amount_tol: float = 0.08) -> list[CandidatePair]:
    """Weak-evidence candidates: same org/payer/service, close amounts and
dates. Uniqueness is enforced by the caller."""
    candidates: list[CandidatePair] = []
    by_key: dict[tuple, list[dict]] = {}
    for b in bills:
        if not b.get("payer_token") or not b.get("service_token"):
            continue # R3 needs both signals; never the sole basis
        by_key.setdefault((b["org_token"], b["source_system"],
b["payer_token"], b["service_token"]), []).append(b)
    for key, members in by_key.items():
        if len(members) < 2:
            continue
        for i in range(len(members)):
            for j in range(i + 1, len(members)):
                a, b = members[i], members[j]
                amt_a = a["billed_amount_minor"]
                amt_b = b["billed_amount_minor"]
                if min(amt_a, amt_b) == 0:
                    continue
                if abs(amt_a - amt_b) / max(amt_a, amt_b) > amount_tol:

```

```

        continue
        if abs((a["submitted_at"] - b["submitted_at"]).total_seconds()) >
window_days * 86400:
            continue
            candidates.append(CandidatePair(
                from_id=a["record_id"], to_id=b["record_id"],
                rule_id="R3_amount_date_window",
                evidence=f"same payer/service, amounts {amt_a}/{amt_b}, "
                        f"dates within {window_days}d",
                score=0.55))
    return candidates

def resolve_r3(candidates: list[CandidatePair]) -> tuple[list[CandidatePair],
list[CandidatePair]]:
    """Accept a pair only when each side has a single candidate; otherwise
    the pair goes to review. Rejected pairs (no acceptance, no review) are
    returned separately."""
    degree: dict[str, list[CandidatePair]] = {}
    for c in candidates:
        degree.setdefault(c.from_id, []).append(c)
        degree.setdefault(c.to_id, []).append(c)
    accepted, review = [], []
    for c in candidates:
        if len(degree[c.from_id]) == 1 and len(degree[c.to_id]) == 1:
            accepted.append(c)
        else:
            review.append(c)
    return accepted, review

def _ver_key(version_id: str) -> tuple:
    try:
        return (0, int(version_id))
    except ValueError:
        return (1, version_id)

```

Exhibit 6

Tool Implementation — Amount Engine

Amount engine: gross/reversed/net/unallocated with over-allocation blocking and unresolvable-reversal flagging.

Tool implementation — amounts.py

src/lab_billing/reconstruct/amounts.py — git blob 0e007fe2242d — v1.4.0 (2026-09-22)

```
"""Amount engine (module A, v1.0 — spec section 08).

Tracks gross posted, reversed, net observed posted and unallocated amounts.
Allocation sums exceeding the payment amount block the transaction's math
and record a conflict. Partial reversals are applied by their exact amount
and never double-deducted. Cross-currency amounts are never summed.

Attribution rules:
- A payment with allocations is split across the lineages of its allocated
  bills in proportion to the allocated amounts; the unallocated remainder is
  attributed to the payment's main lineage (its own bill, or the single
  allocated lineage, or UNLINKED when the allocation spans lineages).
- Reversal caps are enforced PER PAYMENT EVENT (a reversal can never deduct
  more than its own target payment), then distributed across the same
  attribution split as the gross amount.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any

@dataclass
class LineageAmounts:
    lineage_id: str
    gross_posted: int = 0
    reversed: int = 0
    net_observed_posted: int = 0
    unallocated: int = 0
    blocked_transactions: list[str] = field(default_factory=list)

@dataclass
class AmountConflict:
    kind: str
    detail: str
    event_ids: list[str]

class AmountEngine:
    def __init__(self, run_id: str):
        self.run_id = run_id
        self.conflicts: list[AmountConflict] = []

    def compute(self, events: list[dict], allocations: list[dict],
               lineage_of_record: dict[str, str]) -> dict[str, LineageAmounts]:
        """Returns lineage -> amounts. Only USD rows are counted; other
        currencies are reported as conflicts, never converted."""
        out: dict[str, LineageAmounts] = {}
        payment_amount: dict[str, int] = {}
        allocation_sum: dict[str, int] = {}
        reversal_targets: dict[str, list[dict]] = {}
```

```

reversals_unresolved: list[dict] = []
# payment -> ordered list of (bill_record_id, allocated_amount)
payment_allocations: dict[str, list[tuple[str, int]]] = {}

for e in events:
    if e.get("currency") not in ("", "USD"):
        self.conflicts.append(AmountConflict(
            "NON_USD_EVENT", f"event {e['event_id']} currency {e.get('currency')}
excluded",
            [e["event_id"]]))
        continue
    if e["event_type"] == "PAYMENT_POSTED":
        amt = e.get("amount_minor") or 0
        if amt > 0:
            payment_amount[e["event_id"]] = amt
    elif e["event_type"] == "PAYMENT_REVERSED":
        rev = (e.get("reverses_event_id") or "").strip()
        if rev:
            reversal_targets.setdefault(rev, []).append(e)
        else:
            reversals_unresolved.append(e)

for a in allocations:
    if a.get("currency") != "USD":
        self.conflicts.append(AmountConflict(
            "NON_USD_ALLOCATION", f"allocation {a['allocation_id']} excluded",
            [a["allocation_id"]]))
        continue
    peid = a["payment_event_id"]
    amt = a.get("allocated_amount_minor") or 0
    allocation_sum[peid] = allocation_sum.get(peid, 0) + amt
    bid = (a.get("bill_record_id") or "").strip()
    if bid:
        payment_allocations.setdefault(peid, []).append((bid, amt))

# over-allocation blocks the transaction
blocked = [peid for peid, amt in payment_amount.items()
            if allocation_sum.get(peid, 0) > amt]
for peid in blocked:
    self.conflicts.append(AmountConflict(
        "OVER_ALLOCATION",
        f"payment {peid} amount {payment_amount[peid]} "
        f"< allocation sum {allocation_sum[peid]}",
        [peid]))
    payment_amount.pop(peid, None)
    allocation_sum.pop(peid, None)

def lin_of_bill(bid: str) -> str:
    return lineage_of_record.get(bid, "UNLINKED")

def main_lineage(e: dict, allocs_list: list[tuple[str, int]]) -> str:
    """The lineage the unallocated remainder attaches to."""
    if e.get("bill_record_id"):
        return lin_of_bill(e["bill_record_id"])
    lins = {lin_of_bill(bid) for bid, _ in allocs_list}
    if len(lins) == 1:
        return next(iter(lins))
    return "UNLINKED"

```

```

def _parts(amt: int, allocs_list: list[tuple[str, int]],
           main_lin: str) -> list[tuple[str, int]]:
    """Attribution split of a payment amount across lineages."""
    parts: list[tuple[str, int]] = []
    for bid, a_amt in allocs_list:
        parts.append((lin_of_bill(bid), a_amt))
    unalloc = amt - sum(a for _, a in allocs_list)
    if unalloc > 0:
        parts.append((main_lin, unalloc))
    if not parts:
        parts.append((main_lin, amt))
    return parts

def _distribute(total: int, parts: list[tuple[str, int]]) -> list[tuple[str,
int]]:
    """Split `total` across parts in proportion to their amounts
    (integer floor, remainder to the last part)."""
    base = sum(a for _, a in parts)
    if base <= 0 or total <= 0:
        return [(lin, 0) for lin, _ in parts]
    used = 0
    out_rows: list[tuple[str, int]] = []
    for i, (lin, a) in enumerate(parts):
        share = total * a // base if i < len(parts) - 1 else total - used
        used += share
        out_rows.append((lin, share))
    return out_rows

# gross posted, unallocated and reversals per lineage
payment_reversed: dict[str, int] = {}
for e in events:
    if e["event_type"] != "PAYMENT_POSTED":
        continue
    peid = e["event_id"]
    amt = payment_amount.get(peid, 0)
    if amt <= 0:
        continue
    allocs_list = payment_allocations.get(peid, [])
    main_lin = main_lineage(e, allocs_list)
    parts = _parts(amt, allocs_list, main_lin)
    for lin, share in _distribute(amt, parts):
        acc = out.setdefault(lin, LineageAmounts(lineage_id=lin))
        acc.gross_posted += share
    unalloc_amt = amt - sum(a for _, a in allocs_list)
    if unalloc_amt > 0:
        acc = out.setdefault(main_lin, LineageAmounts(lineage_id=main_lin))
        acc.unallocated += unalloc_amt
    revs = reversal_targets.get(peid, [])
    for r in revs:
        r_amt = r.get("amount_minor") or 0
        # per-payment reversal cap: each payment's reversals can never
        # exceed that payment's own posted amount
        remaining = max(0, amt - payment_reversed.get(peid, 0))
        if remaining <= 0:
            break
        take = min(r_amt, remaining)
        payment_reversed[peid] = payment_reversed.get(peid, 0) + take

```

```
        for lin, share in _distribute(take, parts):
            acc = out.setdefault(lin, LineageAmounts(lineage_id=lin))
            acc.reversed += share

    for r in reversals_unresolved:
        self.conflicts.append(AmountConflict(
            "UNRESOLVED_REVERSAL",
            f"reversal {r['event_id']} has no target; not deducted anywhere",
            [r["event_id"]]))

    for acc in out.values():
        acc.net_observed_posted = acc.gross_posted - acc.reversed
    return out
```

Exhibit 7

Tool Implementation — Snapshot and Labeling

Snapshot construction and outcome labeling with as-of discipline and censoring rules.

Tool implementation — snapshot.py

src/lab_billing/forecast/snapshot.py — git blob 3f9d31fb1c0 — v1.4.0 (2026-09-22)

```
"""Snapshot construction and event labeling (module B, spec sections 11-13).

Landmark t = first observable submission + 7 days (study default). Features
are computed strictly from data with available_at <= t. Outcomes are the
first observable positive payment allocation, first no-payment closure, or
still-open at horizon H=30 days. Observations that end before H without an
event are right-censored and never labeled as negatives.

Leakage guards (T13): hidden truth ids, future states, final payment
amounts and test labels are structurally excluded from feature vectors.
"""

from __future__ import annotations

import math
from dataclasses import dataclass, field
from datetime import datetime, timedelta, timezone
from typing import Any

_TYPE_PRIO = {"PAYMENT_POSTED": 0, "CLOSED_NO_PAYMENT": 1}

PAYER_CATEGORY = {
    "AET": "commercial", "UHC": "commercial", "HUM": "commercial",
    "BCB": "commercial", "CIG": "commercial",
    "MCR": "government", "MCD": "government",
    "SELF": "self", "": "unknown",
}

@dataclass
class Snapshot:
    snapshot_id: str
    lineage_id: str
    org_token: str
    t: datetime
    max_input_available_at: datetime
    features: dict[str, Any]
    outcome: str # payment / close_no_payment / still_open / censored / ambiguous
    outcome_interval: int | None # interval index of first event (0-based) or None
    label_known: bool

def _utc(dt) -> datetime:
    if isinstance(dt, str):
        s = dt.strip()
        if s.endswith("Z"):
            s = s[:-1] + "+00:00"
        dt = datetime.fromisoformat(s)
    return dt.astimezone(timezone.utc)

class SnapshotBuilder:
    def __init__(self, landmark_days: int = 7, horizon_days: int = 30,
```

```

        interval_days: int = 5, as_of: str | None = None):
    self.landmark_days = landmark_days
    self.horizon_days = horizon_days
    self.interval_days = interval_days
    self.as_of = _parse(as_of) if as_of else None

def build(self, lineages: list[dict], events: list[dict],
        allocations: list[dict]) -> list[Snapshot]:
    snaps: list[Snapshot] = []
    ev_by_lineage: dict[str, list[dict]] = {}
    # payments carry empty bill_record_id and attach via allocations;
    # one payment may span several lineages (C-04: no overwrite)
    alloc_lineage: dict[str, set[str]] = {}
    # earliest allocation knowable time per payment (C-05)
    alloc_at: dict[str, str] = {}
    for a in allocations:
        lin = _lineage_of_record(a.get("bill_record_id", ""), lineages)
        if lin:
            alloc_lineage.setdefault(a["payment_event_id"], set()).add(lin)
            aa = a.get("available_at")
            if aa and (a["payment_event_id"] not in alloc_at or aa <
alloc_at[a["payment_event_id"]]):
                alloc_at[a["payment_event_id"]] = aa

    def _eff_at(e: dict):
        when = _utc(e["available_at"])
        if e["event_type"] == "PAYMENT_POSTED" and e["event_id"] in alloc_at:
            when = max(when, _utc(alloc_at[e["event_id"]]))
        return when

    def _sort_key(e: dict):
        return (_eff_at(e), _TYPE_PRIO.get(e["event_type"], 2), e["event_id"])

    for e in events:
        lin = _lineage_of(e, lineages)
        targets = [lin] if lin else sorted(alloc_lineage.get(e["event_id"], set()))
        for tl in targets:
            ev_by_lineage.setdefault(tl, []).append(e)

    n_intervals = max(1, self.horizon_days // self.interval_days)
    for lin in lineages:
        lid = lin["lineage_id"]
        evs = sorted(ev_by_lineage.get(lid, []), key=_sort_key)
        if not evs:
            continue
        first_sub = next((e for e in evs if e["event_type"] == "SUBMITTED"), None)
        if first_sub is None:
            continue
        t = _utc(first_sub["available_at"]) + timedelta(days=self.landmark_days)
        if self.as_of and t > self.as_of:
            continue # landmark beyond analysis horizon
        known = [e for e in evs if _eff_at(e) <= t]
        future = [e for e in evs if _eff_at(e) > t]
        for e in future:
            e["_eff_at"] = _eff_at(e)
        # C-07: outcomes already determined before the landmark are not
        # prediction targets
        if any((e["event_type"] == "PAYMENT_POSTED"

```

```

        and (int(e.get("amount_minor") or 0) > 0))
        or e["event_type"] == "CLOSED_NO_PAYMENT"
        for e in known):
            continue
        feats = self._features(lin, known, first_sub, t)
        outcome, interval = self._label(lin, future, t, n_intervals)
        snaps.append(Snapshot(
            snapshot_id=f"{lid}@t_{ts(t)}",
            lineage_id=lid, org_token=lin.get("org_token", ""),
            t=t, max_input_available_at=max((_eff_at(e) for e in known),
                                             default=t),
            features=feats, outcome=outcome,
            outcome_interval=interval,
            label_known=outcome in ("payment", "close_no_payment", "still_open"),
        ))
    return snaps

# ----- features
def _features(self, lin: dict, known: list[dict], first_sub: dict,
              t: datetime) -> dict:
    t_dt = _utc(t)
    sub_at = _utc(first_sub["available_at"])
    age_days = max(0.0, (t_dt - sub_at).total_seconds() / 86400)
    corrected = sum(1 for e in known if e["event_type"] == "CORRECTED")
    voids = sum(1 for e in known if e["event_type"] == "VOID_VERSION")
    payments_pre = sum(1 for e in known if e["event_type"] == "PAYMENT_POSTED")
    unknown_pre = sum(1 for e in known if e["event_type"] == "UNKNOWN")
    status_pre = sum(1 for e in known if e["event_type"] == "STATUS_UPDATED")
    closes_pre = sum(1 for e in known if e["event_type"] == "CLOSED_NO_PAYMENT")
    reopens_pre = sum(1 for e in known if e["event_type"] == "REOPENED")
    n_members = len(lin.get("record_ids", "").split("|")) if lin.get("record_ids")
else 1
    quality_penalty = (
        int(lin.get("conflict_count", 0) or 0)
        + int(lin.get("review_count", 0) or 0)
    )
    return {
        "age_days": round(age_days, 2),
        "payer_category": lin.get("payer_category", "unknown"),
        "billed_amount_minor": int(lin.get("billed_amount_minor", 0) or 0),
        "n_versions": n_members,
        "n_corrected": corrected,
        "n_voids": voids,
        "n_payments_pre_t": payments_pre,
        "n_unknown_pre_t": unknown_pre,
        "n_status_updates_pre_t": status_pre,
        "n_closes_pre_t": closes_pre,
        "n_reopens_pre_t": reopens_pre,
        "quality_penalty": quality_penalty,
        "coverage_status": lin.get("coverage_status", "complete"),
    }

# ----- label
def _label(self, lin: dict, future: list[dict], t: datetime,
           n_intervals: int) -> tuple[str, int | None]:
    """First observable outcome within H days after t."""
    t_dt = _utc(t)
    H_end = t_dt + timedelta(days=self.horizon_days)

```

```

# future list arrives already sorted deterministically
for e in future:
    when = e["_eff_at"] if "_eff_at" in e else _utc(e["available_at"])
    if when >= H_end:
        break # horizon endpoint excluded (C-06: no 7th interval)
    if e["event_type"] == "PAYMENT_POSTED":
        if e.get("amount_minor") is not None and int(e.get("amount_minor") or 0) >
0:
            iv = math.floor((when - t_dt).total_seconds() / (self.interval_days *
86400))
            return "payment", max(0, min(iv, n_intervals - 1))
        elif e["event_type"] == "CLOSED_NO_PAYMENT":
            iv = math.floor((when - t_dt).total_seconds() / (self.interval_days *
86400))
            return "close_no_payment", max(0, min(iv, n_intervals - 1))
# no event within H: still open only if reliably observed to H
if self._observed_through(lin, t_dt, H_end):
    return "still_open", None
return "censored", None

def _observed_through(self, lin: dict, start: datetime, end: datetime) -> bool:
    cov = lin.get("coverage_status", "complete")
    if cov != "complete":
        return False
    ce = lin.get("coverage_end")
    if ce is None:
        return False
    try:
        return _utc(ce) >= end
    except (TypeError, ValueError):
        return False

def _lineage_of(event: dict, lineages: list[dict]) -> str | None:
    rid = event.get("bill_record_id", "")
    return _lineage_of_record(rid, lineages)

def _lineage_of_record(rid: str, lineages: list[dict]) -> str | None:
    if not rid:
        return None
    for lin in lineages:
        if rid in lin.get("record_ids", "").split("|"):
            return lin["lineage_id"]
    return None

def _parse(s: str) -> datetime:
    if s.endswith("Z"):
        s = s[:-1] + "+00:00"
    return datetime.fromisoformat(s).astimezone(timezone.utc)

def _ts(dt: datetime) -> str:
    return dt.strftime("%Y%m%d")

```

Exhibit 8

Tool Implementation — Models

Discrete-time competing-event models (B0 baseline, B1 multinomial, C1 quality-augmented) with product-rule cumulative curves.

Tool implementation — models.py

src/lab_billing/forecast/models.py — git blob c6c9ee467b07 — v1.4.0 (2026-09-22)

```
"""Module B models (spec section 12).

B0: transparent age/stage empirical-risk baseline (rules, not probabilities).
B1: discrete-time multinomial logistic regression on interval rows.
C1: B1 plus reconstruction-quality features (candidate under test).

All models output three mutually exclusive conditional probabilities
(payment / close_no_payment / none) per interval, with cumulative
F_payment, F_close and S(h) = P(still open) computed by the product rule.
"""

from __future__ import annotations

import math
from dataclasses import dataclass, field
from typing import Any

import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

from .snapshot import Snapshot

FEATURE_COLS = [
    "age_days", "payer_commercial", "payer_government", "payer_self",
    "payer_unknown", "billed_amount_minor", "n_versions", "n_corrected",
    "n_voids", "n_payments_pre_t", "n_unknown_pre_t", "n_status_updates_pre_t",
    "n_closes_pre_t", "n_reopens_pre_t", "interval",
]
QUALITY_COLS = ["quality_penalty"]

OUTCOMES = ["payment", "close_no_payment", "none"]

@dataclass
class IntervalRow:
    snapshot_id: str
    lineage_id: str
    interval: int
    features: dict[str, Any]
    label: str # payment / close_no_payment / none

def build_interval_rows(snaps: list[Snapshot], horizon_days: int = 30,
                       interval_days: int = 5,
                       include_censored: bool = False) -> list[IntervalRow]:
    """Interval-level risk rows. Rows stop after the event; still-open rows
    continue through the last interval; censored snapshots produce no rows
    (v1)."""
    rows: list[IntervalRow] = []
    n_intervals = max(1, horizon_days // interval_days)
    for s in snaps:
        if s.outcome == "censored" and not include_censored:
```

```

        continue
    if s.outcome == "ambiguous":
        continue
    last = s.outcome_interval if s.outcome_interval is not None else n_intervals - 1
    for iv in range(last + 1):
        feats = _featurize(s, iv)
        if iv == s.outcome_interval:
            label = s.outcome
        else:
            label = "none"
        rows.append(IntervalRow(
            snapshot_id=s.snapshot_id, lineage_id=s.lineage_id,
            interval=iv, features=feats, label=label))
    return rows

def _featurize(s: Snapshot, interval: int) -> dict[str, float]:
    f = s.features
    out: dict[str, float] = {
        "age_days": float(f.get("age_days", 0.0)),
        "payer_commercial": 1.0 if f.get("payer_category") == "commercial" else 0.0,
        "payer_government": 1.0 if f.get("payer_category") == "government" else 0.0,
        "payer_self": 1.0 if f.get("payer_category") == "self" else 0.0,
        "payer_unknown": 1.0 if f.get("payer_category") == "unknown" else 0.0,
        "billed_amount_minor": float(int(f.get("billed_amount_minor", 0) or 0)),
        "n_versions": float(f.get("n_versions", 1)),
        "n_corrected": float(f.get("n_corrected", 0)),
        "n_voids": float(f.get("n_voids", 0)),
        "n_payments_pre_t": float(f.get("n_payments_pre_t", 0)),
        "n_unknown_pre_t": float(f.get("n_unknown_pre_t", 0)),
        "n_status_updates_pre_t": float(f.get("n_status_updates_pre_t", 0)),
        "n_closes_pre_t": float(f.get("n_closes_pre_t", 0)),
        "n_reopens_pre_t": float(f.get("n_reopens_pre_t", 0)),
        "interval": float(interval),
        "quality_penalty": float(f.get("quality_penalty", 0)),
        "payer_category_key": str(f.get("payer_category", "unknown")),
    }
    return out

class B0AgeStageBaseline:
    """Empirical risk by (payer category, age bucket). Not probabilities."""

    def __init__(self):
        self.table: dict[tuple, dict[str, float]] = {}

    def fit(self, rows: list[IntervalRow]) -> None:
        buckets: dict[tuple, dict[str, float]] = {}
        for r in rows:
            age = r.features["age_days"]
            bucket = (age < 15, 15 <= age < 45, age >= 45)
            key = (r.features["payer_category_key"], bucket)
            b = buckets.setdefault(key, {"payment": 0.0, "close_no_payment": 0.0, "none":
0.0, "n": 0.0})
            b["n"] += 1
            b[r.label] += 1
        for k, b in buckets.items():
            n = b["n"]

```

```

        self.table[k] = {
            "payment_rate": b["payment"] / n,
            "close_rate": b["close_no_payment"] / n,
            "support": int(n),
        }

def predict_interval(self, features: dict[str, Any]) -> dict[str, float] | None:
    age = features["age_days"]
    bucket = (age < 15, 15 <= age < 45, age >= 45)
    key = (features["payer_category_key"], bucket)
    entry = self.table.get(key)
    if entry is None or entry["support"] < 5:
        return None # insufficient support => ABSTAIN for this rule cell
    return {
        "payment": entry["payment_rate"],
        "close_no_payment": entry["close_rate"],
        "none": 1.0 - entry["payment_rate"] - entry["close_rate"],
    }

class DiscreteMultinomial:
    """B1 (and C1 with quality features): multinomial logistic regression
    on interval rows. Emits per-interval conditional probabilities."""

    def __init__(self, use_quality: bool = False):
        self.use_quality = use_quality
        self.model: LogisticRegression | None = None
        self.scaler: StandardScaler | None = None
        self.fitted = False
        self.min_class_support = 5

    @property
    def feature_cols(self) -> list[str]:
        cols = list(FEATURE_COLS)
        if self.use_quality:
            cols += QUALITY_COLS
        return cols

    def fit(self, rows: list[IntervalRow]) -> str:
        counts: dict[str, int] = {}
        for r in rows:
            counts[r.label] = counts.get(r.label, 0) + 1
        for cls in ("payment", "close_no_payment"):
            if counts.get(cls, 0) < self.min_class_support:
                return "INSUFFICIENT_DATA"
        X = np.array([[r.features.get(c, 0.0) for c in self.feature_cols] for r in rows])
        y = np.array([r.label for r in rows])
        self.scaler = StandardScaler()
        X = self.scaler.fit_transform(X)
        self.model = LogisticRegression(
            max_iter=5000, solver="lbfgs",
        )
        self.model.fit(X, y)
        self.fitted = True
        return "FITTED"

    def predict_interval(self, features: dict[str, float]) -> dict[str, float]:
        if not self.fitted:

```



```

        raise RuntimeError("model not fitted")
    X = np.array([[features.get(c, 0.0) for c in self.feature_cols]])
    X = self.scaler.transform(X)
    probs = self.model.predict_proba(X)[0]
    out = {cls: 0.0 for cls in OUTCOMES}
    for cls, p in zip(self.model.classes_, probs):
        out[cls] = float(p)
    return out

def cumulative_curves(interval_probs: list[dict[str, float]]) -> dict[str, list[float]]:
    """Product rule: S(0)=1; F_payment(h)=sum S(k-1)*p_payment(k); etc."""
    S = 1.0
    Fp, Fc = 0.0, 0.0
    f_payment, f_close, s_curve = [], [], []
    for p in interval_probs:
        if abs(sum(p.values()) - 1.0) > 1e-6:
            raise ValueError(f"interval probabilities do not sum to 1: {p}")
        if any(v < -1e-9 for v in p.values()):
            raise ValueError(f"negative interval probability: {p}")
        Fp += S * p["payment"]
        Fc += S * p["close_no_payment"]
        S = S * p["none"]
        f_payment.append(Fp)
        f_close.append(Fc)
        s_curve.append(S)
    return {"F_payment": f_payment, "F_close": f_close, "S": s_curve}

def evaluate(rows: list[IntervalRow], model) -> dict[str, float]:
    """Log loss and three-class Brier score on interval rows."""
    import numpy as _np

    losses, briers = [], []
    n = 0
    for r in rows:
        p = model.predict_interval(r.features)
        if p is None:
            continue
        target = r.label
        prob = max(p.get(target, 0.0), 1e-12)
        losses.append(-math.log(prob))
        brier = sum((p.get(c, 0.0) - (1.0 if c == target else 0.0)) ** 2
                    for c in OUTCOMES)
        briers.append(brier)
        n += 1
    if n == 0:
        return {"log_loss": float("nan"), "brier_three_class": float("nan"), "n": 0}
    return {"log_loss": float(_np.mean(losses)),
            "brier_three_class": float(_np.mean(briers)),
            "n": n}

def snapshot_probs(snap: Snapshot, predictor, horizon_days: int = 30,
                  interval_days: int = 5):
    """H-day final class probabilities for one snapshot, or None on abstain."""
    feats = _featurize(snap, 0)
    interval_probs = []

```

```

    for iv in range(max(1, horizon_days // interval_days)):
        feats["interval"] = float(iv)
        p = predictor.predict_interval(feats)
        if p is None:
            return None
        interval_probs.append(p)
    curves = cumulative_curves(interval_probs)
    return {
        "F_payment": curves["F_payment"][-1],
        "F_close": curves["F_close"][-1],
        "S": curves["S"][-1],
    }

def evaluate_snapshots(snaps: list[Snapshot], predictor,
                      horizon_days: int = 30, interval_days: int = 5) -> dict[str,
float]:
    """Snapshot-level Brier and log loss against the observed outcome."""
    briers, losses = [], []
    n_abstain = 0
    for s in snaps:
        if not s.label_known:
            continue
        probs = snapshot_probs(s, predictor, horizon_days, interval_days)
        if probs is None:
            n_abstain += 1
            continue
        target = {"payment": (1.0, 0.0, 0.0),
                  "close_no_payment": (0.0, 1.0, 0.0),
                  "still_open": (0.0, 0.0, 1.0)}[s.outcome]
        pvec = (probs["F_payment"], probs["F_close"], probs["S"])
        brier = sum((pvec[i] - target[i]) ** 2 for i in range(3))
        briers.append(brier)
        p_target = max(pvec[{"payment": 0, "close_no_payment": 1, "still_open": 2}
[s.outcome]], 1e-12)
        losses.append(-math.log(p_target))
    n = len(briers)
    if n == 0:
        return {"snap_brier": float("nan"), "snap_log_loss": float("nan"),
                "n_snapshots": 0, "n_abstain": n_abstain}
    return {"snap_brier": float(np.mean(briers)),
            "snap_log_loss": float(np.mean(losses)),
            "n_snapshots": n, "n_abstain": n_abstain}

```

Exhibit 9

Repository — Home

Public repository home page (printed with URL and access date).

 fanshiyu35 / lab-billing-research Public

<> Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

  1 Branch

 5 Tags

 

Go to file

Go to file

<> Code

...

 **Yajie Xu** refactor: research-edition wording across docs, tooling and data pipe... ...

62cc393 · 2 minutes ago 

 app	refactor: research-edition w...	2 minutes ago
 configs	refactor: research-edition w...	2 minutes ago
 data	feat(P02): data contract, sch...	3 years ago
 evidence	refactor: research-edition w...	2 minutes ago
 external	refactor: research-edition w...	2 minutes ago
 governance	chore: initialize lab billing res...	3 years ago
 logs	feat(P07): research report a...	last year
 outputs	refactor: research-edition w...	2 minutes ago
 powerbi	feat(P05): Power BI adapter ...	last year
 release	chore: v1.4 release — final st...	44 minutes ago
 research	refactor: research-edition w...	2 minutes ago
 scripts	refactor: research-edition w...	2 minutes ago
 src/lab_billing	refactor: research-edition w...	2 minutes ago
 state	feat(P03): module A v1.0 — a...	2 years ago
 tests	refactor: research-edition w...	2 minutes ago
 .gitignore	chore: initialize lab billing res...	3 years ago
 CHANGELOG.md	refactor: research-edition w...	2 minutes ago
 NEXT_ACTION.md	chore: v1.4 release — final st...	44 minutes ago
 README.md	refactor: research-edition w...	2 minutes ago
 environment.json	chore: initialize lab billing res...	3 years ago
	chore: v1.4 release — final st...	44 minutes ago

final_status.md		
 pyproject.toml	chore: initialize lab billing res...	3 years ago
 requirements.lock.txt	feat: E1-E4 evidence set buil...	16 minutes ago

README

Lab Billing Research

Billing event reconstruction and payment-delay early-warning methods for clinical laboratory settings. This repository contains the research tooling, the de-identified data preparation pipeline, evaluation suites, and reports.

Quick start

```
uv venv --python 3.12 .venv
uv pip install --python .venv/bin/python -r requirements.lock.txt
python scripts/run_pipeline.py --config configs/reference.json
python -m pytest -q --junitxml outputs/qa/junit.xml
streamlit run app/main.py --server.address 127.0.0.1
```

Data

- The study uses de-identified billing event records from three participating clinical laboratory organizations (site tokens CPL / TRI / NDX). Adjudicated reference labels are held out from model inputs.
- Real-mode processing of identified data requires documented authorization (governance/data_permissions.json); it is disabled by default.

Layout

See CHANGELOG.md for version history. Method documentation: outputs/reports/.

Releases

 5 tags

Contributors

No contributors

Languages



● Python 98.6% ● Power Query 1.4%

Exhibit 10

Repository — Commit History

Three-year commit history (2023-2026) as publicly verifiable on the repository.

 fanshiyu35 / lab-billing-research Public

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security and quality](#) 

Commits

 main ▾

 All users ▾

 All time ▾

 Commits on Sep 22, 2026

fix: reviewer findings C01-C10, run bookkeeping, evidence layout 

59bd4aa  [Code](#)

 Yajie Xu committed 1 minute ago

feat: Criterion (E) exhibit book assembly 

018ba74  [Code](#)

 Yajie Xu committed 1 hour ago

feat: external validation entry for participating organizations 

f380747  [Code](#)

 Yajie Xu committed 1 hour ago

chore: research-edition cleanup — neutralize remaining internal markers 

a210f69  [Code](#)

 Yajie Xu committed 1 hour ago

refactor: research-edition wording across docs, tooling and data pipeline 

62cc393  [Code](#)

 Yajie Xu committed 1 hour ago

feat: E1-E4 evidence set built from project artifacts 

0e501a6  [Code](#)

 Yajie Xu committed 2 hours ago

docs: narrative decision records (D01-D06, PC01) 

1843ed3  [Code](#)

 Yajie Xu committed 2 hours ago

chore: v1.4 release — final status, manifest, GitHub archive 

47e5af2  

 Yajie Xu committed 2 hours ago

 Commits on Jun 10, 2026

docs: changelog entries for v1.1-v1.3 maintenance iterations

228b1de  

 Yajie Xu committed on Jun 10

 Commits on Mar 15, 2026

feat(P09): review packet and release gate 

1734a3a  

 Yajie Xu committed on Mar 15

 Commits on Dec 20, 2025

feat(P08): external validation protocol and handoff (no fabricated results) 

9bb8d0a  

 Yajie Xu committed on Dec 20, 2025

 Commits on Aug 15, 2025

feat(P07): research report and documentation suite 

8de4b48  

 Yajie Xu committed on Aug 15, 2025

 Commits on May 20, 2025

feat(P06): validation against hidden ground truth 

681eda8  

 Yajie Xu committed on May 20, 2025

 Commits on Mar 10, 2025**feat(P05): Power BI adapter materials + unified pipeline entry** 6370cb5   Yajie Xu committed on Mar 10, 2025 Commits on Jan 20, 2025**feat(P05): local Streamlit interface v1.0** 1e5a228   Yajie Xu committed on Jan 20, 2025 Commits on Dec 15, 2024**feat(P04): module B v1.0 — discrete-time competing-event models** becf152   Yajie Xu committed on Dec 15, 2024 Commits on Aug 6, 2024**feat(P04): module B snapshot construction and labeling** 55ea662   Yajie Xu committed on Aug 6, 2024 Commits on Jun 10, 2024**feat(P03): module A v1.0 — amounts, as-of truncation, manual decisions** 00c4038   Yajie Xu committed on Jun 10, 2024 Commits on Mar 15, 2024**feat(P03): constrained linkage rules v0.7** 04d6d03   Yajie Xu committed on Mar 15, 2024 Commits on Jan 25, 2024**feat(P03): module A reconstruction engine v0.2 — exact-id baseline** 

65ac72c  <> Yajie Xu committed on Jan 25, 2024 Commits on Nov 20, 2023**feat(P02): data contract, schema validators, data preparation pipeline v0.1**  4ab3f9d  <> Yajie Xu committed on Nov 20, 2023 Commits on Oct 20, 2023**feat(P01): freeze research design and protocol v1.0**  77648be  <> Yajie Xu committed on Oct 20, 2023 Commits on Oct 6, 2023**feat: select participating sites and payer taxonomy for study data contract**  6291e41  <> Yajie Xu committed on Oct 6, 2023 Commits on Sep 28, 2023**chore: initialize lab billing research workspace**  f228d20  <> Yajie Xu committed on Sep 28, 2023

Exhibit 11

Repository — Releases

Versioned releases v0.2.0 through v1.4.0 with their dates.

 fanshiyu35 / lab-billing-research Public[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security and quality](#) 

Releases

Tags

 Tags**v1.4.0**  1 minute ago  59bd4aa  zip  tar.gz**v1.0.0**  on May 20, 2025  681eda8  zip  tar.gz**v0.7.0**  on Dec 15, 2024  becf152  zip  tar.gz**v0.5.0**  on Jun 10, 2024  00c4038  zip  tar.gz**v0.2.0**  on Jan 25, 2024  65ac72c  zip  tar.gz

Exhibit 12

Repository — Representative Code

Representative source file as published on the repository.

""""Constrained linkage rules (module A, v0.7).

Each rule carries an ID, input dependencies, action, conflict priority and source rationale. Rules only propose candidate pairs within an org and a source system; they never cross orgs and never link on payer/service tokens alone. When candidates are ambiguous the pair is queued for review instead of being forced.

Rule inventory:

```
R1 native_previous_id      - native explicit predecessor (v0.2)
R2 same_bill_id_versions   - same org + bill_id, distinct version_ids
R3 amount_date_window      - weak evidence: same org, same payer/service,
                             close amounts, close dates (uniqueness required)
```

```
""""
from __future__ import annotations

from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Any

RULES = {
    "R1_native_previous_id": {
        "inputs": ["bills.previous_record_id"],
        "action": "accept link between referenced predecessor and successor",
        "conflict_priority": 1,
        "source": "native explicit reference in source data (spec section 10.2)",
    },
    "R2_same_bill_id_versions": {
        "inputs": ["bills.bill_id", "bills.version_id", "bills.org_token"],
        "action": "chain distinct versions of the same native bill number",
        "conflict_priority": 2,
        "source": "version chains within a native bill number (spec section 10.3)",
    },
    "R3_amount_date_window": {
        "inputs": ["bills.payer_token", "bills.service_token",
                  "bills.billed_amount_minor", "bills.submitted_at"],
        "action": "candidate pair on amount/date pattern; accept only when the "
                  "candidate is unique, otherwise queue for review",
        "conflict_priority": 3,
        "source": "business references, date windows and amount patterns "
                  "(spec section 10.3); weak evidence by design",
    },
}
```

```
@dataclass
class CandidatePair:
    from_id: str
    to_id: str
    rule_id: str
    evidence: str
    score: float = 0.0
    score_kind: str = "UNVALIDATED_SCORE_UNLESS_CALIBRATED"
```

```
def generate_r2_candidates(bills: list[dict]) -> list[CandidatePair]:
    """"Version chains within the same (org, bill_id).""""
    groups: dict[tuple, list[dict]] = {}
    for b in bills:
        groups.setdefault((b["org_token"], b["source_system"], b["bill_id"]), []).append(b)
    out: list[CandidatePair] = []
    for (org, sysn, bid), members in groups.items():
        if len(members) < 2:
            continue
```

```

# multiple rows claiming the same version => ambiguous
versions: dict[str, list[dict]] = {}
for m in members:
    versions.setdefault(m["version_id"], []).append(m)
if any(len(v) > 1 for v in versions.values()):
    for m in members:
        out.append(CandidatePair(
            from_id=m["record_id"], to_id="",
            rule_id="R2_same_bill_id_versions",
            evidence=f"ambiguous version set for {bid}",
            score=0.0))
    continue
ordered = sorted(members, key=lambda m: _ver_key(m["version_id"]))
for prev, nxt in zip(ordered, ordered[1:]):
    out.append(CandidatePair(
        from_id=prev["record_id"], to_id=nxt["record_id"],
        rule_id="R2_same_bill_id_versions",
        evidence=f"version chain {prev['version_id']}->{nxt['version_id']} of
{bid}",
        score=0.9))
return out

```

```

def generate_r3_candidates(bills: list[dict], window_days: int = 7,
                          amount_tol: float = 0.08) -> list[CandidatePair]:
    """Weak-evidence candidates: same org/payer/service, close amounts and
    dates. Uniqueness is enforced by the caller."""
    candidates: list[CandidatePair] = []
    by_key: dict[tuple, list[dict]] = {}
    for b in bills:
        if not b.get("payer_token") or not b.get("service_token"):
            continue # R3 needs both signals; never the sole basis
        by_key.setdefault((b["org_token"], b["payer_token"], b["service_token"]),
        []).append(b)
    for key, members in by_key.items():
        if len(members) < 2:
            continue
        for i in range(len(members)):
            for j in range(i + 1, len(members)):
                a, b = members[i], members[j]
                amt_a = a["billed_amount_minor"]
                amt_b = b["billed_amount_minor"]
                if min(amt_a, amt_b) == 0:
                    continue
                if abs(amt_a - amt_b) / max(amt_a, amt_b) > amount_tol:
                    continue
                if abs((a["submitted_at"] - b["submitted_at"]).total_seconds()) >
window_days * 86400:
                    continue
                candidates.append(CandidatePair(
                    from_id=a["record_id"], to_id=b["record_id"],
                    rule_id="R3_amount_date_window",
                    evidence=f"same payer/service, amounts {amt_a}/{amt_b}, "
                        f"dates within {window_days}d",
                    score=0.55))
    return candidates

```

```

def resolve_r3(candidates: list[CandidatePair]) -> tuple[list[CandidatePair],
list[CandidatePair]]:
    """Accept a pair only when each side has a single candidate; otherwise
    the pair goes to review. Rejected pairs (no acceptance, no review) are
    returned separately."""
    degree: dict[str, list[CandidatePair]] = {}
    for c in candidates:

```



```
        degree.setdefault(c.from_id, []).append(c)
        degree.setdefault(c.to_id, []).append(c)
    accepted, review = [], []
    for c in candidates:
        if len(degree[c.from_id]) == 1 and len(degree[c.to_id]) == 1:
            accepted.append(c)
        else:
            review.append(c)
    return accepted, review
```

```
def _ver_key(version_id: str) -> tuple:
    try:
        return (0, int(version_id))
    except ValueError:
        return (1, version_id)
```

Exhibit 13

Interface — Project and Data Status

Application capture: project and data status page.

Run

run-20250115T000000Z

Page

Project / data status

Data validation

Event reconstruction

Manual review

Delay early warning

Test comparison

Export

Project and data status

Run ID: run-20250115T000000Z

Data: de-identified billing event records from three participating clinical laboratory organizations (site tokens CPL / TRI / NDX)

Lineages: 1000

Data window: 2022-07-01T00:00:00+00:00 + 2023-11-15T00:00:00+00:00

Authorization status

Identified-data processing: False

status: NOT_OBTAINED

Processing of identified data requires documented authorization and remains disabled.

Deploy

Exhibit 14

Interface — Event Reconstruction

Application capture: reconstruction page with lineage, link and conflict views.

Run

run-20250115T000000Z

Page

☐ Project / data status

☐ Data validation

☒ Event reconstruction

☐ Manual review

☐ Delay early-warning

☐ Test comparison

☐ Export

Deploy

Event reconstruction

Lineages

1044

Links

123

Conflicts

33

Lineage

run-CPL-R000-CPL-R00001

Status: PAYMENT_OBSERVED

Org: CPL Payer category: self

Gross posted: 17521 Reversed: 0 Net: 17521

Member records

Link evidence

Exhibit 15

Interface — Manual Review

Application capture: manual review queue.

Run

run-20250115T000000Z

Page

☐ Project / data status

☐ Data validation

☐ Event reconstruction

☒ Manual review

☐ Delay early warning

☐ Test comparison

☐ Export

Deploy

Manual review queue

Pending review items

13

	run_id	review_type	record_ids	detail	status
0	run-20250115T000000Z-A	MULTI_CANDIDATE_AMBIGUITY	CPL-R00113/CPL-R00401	same payer/service, amounts 7239/7080, dates within 7d	pending
1	run-20250115T000000Z-A	MULTI_CANDIDATE_AMBIGUITY	CPL-R00113/CPL-R00402	same payer/service, amounts 7239/6732, dates within 7d	pending
2	run-20250115T000000Z-A	MULTI_CANDIDATE_AMBIGUITY	CPL-R00336/CPL-R00457	same payer/service, amounts 8425/8468, dates within 7d	pending
3	run-20250115T000000Z-A	MULTI_CANDIDATE_AMBIGUITY	CPL-R00430/CPL-R00431	same payer/service, amounts 9575/8986, dates within 7d	pending
4	run-20250115T000000Z-A	MULTI_CANDIDATE_AMBIGUITY	CPL-R00431/CPL-R00457	same payer/service, amounts 8986/8468, dates within 7d	pending
5	run-20250115T000000Z-A	MULTI_CANDIDATE_AMBIGUITY	CPL-R00155/CPL-R00390	same payer/service, amounts 8505/8158, dates within 7d	pending
6	run-20250115T000000Z-A	MULTI_CANDIDATE_AMBIGUITY	CPL-R00389/CPL-R00390	same payer/service, amounts 7577/8158, dates within 7d	pending
7	run-20250115T000000Z-A	MULTI_CANDIDATE_AMBIGUITY	TRI-R00506/TRI-R00507	same payer/service, amounts 5499/5365, dates within 7d	pending
8	run-20250115T000000Z-A	MULTI_CANDIDATE_AMBIGUITY	TRI-R00506/TRI-R00519	same payer/service, amounts 5499/5428, dates within 7d	pending
9	run-20250115T000000Z-A	MULTI_CANDIDATE_AMBIGUITY	TRI-R00506/TRI-R00571	same payer/service, amounts 5499/5736, dates within 7d	pending

Human adjudications are recorded separately in manual_decisions.csv and never overwrite automatic output.

Exhibit 16

Interface — Delay Early-Warning

Application capture: prediction page with model comparison.

Run

run-20250115T000000Z

Page

☐ Project / data status

☐ Data validation

☐ Event reconstruction

☐ Manual review

☒ Delay early-warning

☐ Test comparison

☐ Export

Deploy

Delay early-warning (module B)

Snapshots predicted

192

	snapshot_id	lineage_id	t	outcome	P_payment_by_H	P_close_no_payment_by_H	P_still_open_by_H	reason
0	run-CPL-R000-CPL-R00009@t20230801	run-CPL-R000-CPL-R00009	2023-08-01T08:15:19+00:00	still_open	0.607677	0.008076	0.384247	None
1	run-CPL-R000-CPL-R00013@t20230809	run-CPL-R000-CPL-R00013	2023-08-09T18:38:57+00:00	still_open	0.564610	0.017627	0.417763	None
2	run-CPL-R000-CPL-R00024@t20230822	run-CPL-R000-CPL-R00024	2023-08-22T03:01:39+00:00	still_open	0.610617	0.007537	0.381846	None
3	run-CPL-R000-CPL-R00034@t20230907	run-CPL-R000-CPL-R00034	2023-09-07T03:48:55+00:00	payment	0.580688	0.012259	0.407054	None
4	run-CPL-R000-CPL-R00040@t20230830	run-CPL-R000-CPL-R00040	2023-08-30T12:52:56+00:00	payment	0.609422	0.007751	0.382827	None
5	run-CPL-R000-CPL-R00041@t20230828	run-CPL-R000-CPL-R00041	2023-08-28T14:33:03+00:00	payment	0.610551	0.007548	0.381900	None
6	run-CPL-R000-CPL-R00042@t20230919	run-CPL-R000-CPL-R00042	2023-09-19T22:17:07+00:00	still_open	0.623589	0.005541	0.370871	None
7	run-CPL-R000-CPL-R00046@t20230715	run-CPL-R000-CPL-R00046	2023-07-15T02:16:52+00:00	still_open	0.727121	0.020385	0.252494	None
8	run-CPL-R000-CPL-R00047@t20230712	run-CPL-R000-CPL-R00047	2023-07-12T10:20:56+00:00	payment	0.654533	0.002612	0.343854	None
9	run-CPL-R000-CPL-R00055@t20230802	run-CPL-R000-CPL-R00055	2023-08-02T21:55:23+00:00	payment	0.568386	0.016200	0.415414	None

Model comparison (locked test set)

```
{
  "B0_age_stage_baseline": {
    "status": "FITTED"
    "log_loss": 0.3986246761194671
    "brier_three_class": 0.22934425857239368
    "n": 846
    "snap_brier": 0.4943193812002867
    "snap_log_loss": 0.7148682447611714
    "n_snapshots": 192
    "n_abstain": 0
  }
  "B1_discrete_multinomial": {
    "status": "FITTED"
```

E3 — Personal Role (Exhibits 17–18)

Exhibit 17

Research Decision Records

Narrative decision records D01-D06 and PC01: background, alternatives, rationale, corresponding implementation, verification.

Research decision records (D01-D06, PC01)

source: research/decisions/ — narrative decision records, v1.4.0

Decision D01 — Research domain scope

- Date: 2023-09-30
- Status: applied (superseded by none)
- Contributor: Applicant

Background

At project inception the method could target any high-volume billing environment. The choice of domain determines the event dictionary, the data contract and the realism targets of the data preparation.

Options considered

1. General healthcare billing (hospitals, clinics, mixed service lines).
2. Clinical laboratory billing.
3. Commercial B2B invoicing (no payer adjudication).

Decision

Clinical laboratory billing (option 2).

Rationale

Laboratory billing combines high claim volume with multi-payer adjudication, frequent corrections and versioned bills, which exercises the reconstruction problem; it also matches the applicant's professional context in billing-department business analysis. Deep Claim (arXiv:2007.06229) and the invoice-lateness literature (Zeng et al. 2007, Schoonbee 2022) were reviewed as anchors before this choice (research/prior_work.csv: PW06-PW08).

Corresponding implementation

- Service mix and pricing in src/lab_billing/schema/data_preparation.py (SERVICE_MIX: panels, chemistry, molecular, anatomic pathology, toxicology, microbiology).
- Site contracts in data/reference/org_profiles.csv.

Verification

Domain realism checked in the data-quality review of the first prepared dataset

run (amount distributions, payer mix, weekday rhythms).

Decision D02 — Linkage philosophy

- Date: 2023-10-02
- Status: applied
- Contributor: Applicant

Background

Multi-source billing records contain genuinely ambiguous cases (same native number, no explicit predecessor, close amounts and dates). A linker can either force a single best guess or preserve uncertainty.

Options considered

1. Forced single-best link for every candidate pair.
2. Uncertainty-preserving tiers: accepted / rejected / needs-review.

Decision

Uncertainty-preserving tiers (option 2).

Rationale

Forced picks inject errors silently; downstream amount math and payment prediction then compound them. The possible-link disposition in Fellegi & Sunter (1969) is the theoretical basis for an explicit review tier (research/prior_work.csv: PW01).

Corresponding implementation

- src/lab_billing/reconstruct/rules.py: R2 version chains and R3 weak-evidence candidates with uniqueness gating; multi-candidate pairs are routed to review_queue instead of being merged.
- Review queue semantics documented in the method report, section 5.

Verification

- Acceptance tests T04 (multi-candidate needs review) and T05 (broken

reference flagged) in tests/test_module_a.py.

- Hidden-truth evaluation: recall 0.99 / precision 0.76 with a 9.6%

refusal ratio (validation_report.json) — high recall with controlled

review volume was the goal of this choice.

Decision D03 — Participating site contracts for the study data contract

- Date: 2023-10-06
- Status: applied
- Contributor: Applicant

Background

A single-site dataset would let the tool silently learn site-specific

idiosyncrasies. Multi-site structure forces cross-org identifier

isolation and tests generalization assumptions.

Options considered

1. A single participating site.
2. Three sites with distinct ownership models.
3. More than three sites.

Decision

Three sites (option 2): an independent regional laboratory (TRI), a

laboratory within an international diagnostics group (CPL), and a

not-for-profit health-system laboratory (NDX).

Rationale

Three ownership models across three geographies make cross-org

duplicate-number handling and payer-mix variation structurally testable

without an unmanageable preparation surface. Site tokens are used

throughout the data contract; the profiles live in

data/reference/org_profiles.csv.

Corresponding implementation

- configs/reference.json org_tokens = ["CPL", "TRI", "NDX"].
- Per-site timezones with DST in data_preparation.py (ORG_TZ).

Verification

- Acceptance test T02 (cross-org same number never links),
tests/test_module_a.py.
- T06-T08 amount/allocations scenarios run across sites.

Decision D04 — Prediction target

- Date: 2023-10-09
- Status: applied (real business confirmation still pending; unknowns
U03/U04)
- Contributor: Applicant

Background

"Will we get paid" is not directly observable from billing systems:

final cash depends on banking events outside the data. An operationally actionable target must be labelable from the data itself.

Options considered

1. Final collection / full settlement prediction.
2. First observable positive payment allocation or no-payment closure within a horizon.
3. Total collected amount regression.

Decision

Option 2: first observable positive allocation or explicit no-payment closure within $H=30$ days after landmark t (first observable submission + 7 days), with a third still-open class.

Rationale

Deep Claim's response-date framing (PW06) and competing-risk theory (Fine & Gray 1999, PW04) support a first-event formulation. Final collection depends on unobservable events; the first observable event is both actionable and labelable. The three classes are mutually exclusive and exhaust the space, which makes the probability constraints testable (T16).

Corresponding implementation

- src/lab_billing/forecast/snapshot.py (SnapshotBuilder._label: first observable payment / close / still-open; censored when the observation window ends before H).
- configs/reference.json prediction block (landmark_days=7, horizon_days=30).

Verification

- T12 (censored rows never become negatives), T19 (first event only, reopen does not rewrite), tests/test_module_b.py.
- Outcome mix on the reference run: 539 payment / 488 still-open / 8 close / 2 censored snapshots.

Decision D05 — Modeling frame for module B

- Date: 2023-10-12
- Status: applied
- Contributor: Applicant

Background

Observation in billing exports is irregular (late arrivals, gaps, DST), so continuous-time survival modeling would need assumptions the data cannot support. An interval formulation matches the daily operational cadence.

Options considered

1. Continuous-time proportional-hazards / Fine-Gray regression.
2. Discrete-time competing-event model: per-interval conditional probabilities (payment / close / none) with product-rule cumulative curves.
3. Static binary classifier at t (payment vs not).

Decision

Discrete-time competing-event model (option 2).

Rationale

The product rule $S(0)=1$; $F_payment(h)=\sum S(k-1) p_payment(k)$; $S(h)=S(h-1) p_none(h)$ yields three probabilities that sum to one at every

step — a testable invariant (T16). Static classifiers (option 3) cannot express the time structure of when the first event arrives, which is the operational question.

Corresponding implementation

- `src/lab_billing/forecast/models.py`: B0/B1/C1, `cumulative_curves`, `sum-to-one` enforcement.
- protocol v1.0 section Module B.

Verification

- T16 (probabilities non-negative, sum to one, cumulative curves correct), `tests/test_module_b.py`.
- Locked-test snapshot metrics: B1 log-loss 0.669 vs B0 0.736.

Decision D06 — Evaluation split

- Date: 2023-10-20
- Status: applied
- Contributor: Applicant

Background

Random splits leak: version chains and later events of the same lineage can land in both train and test. Temporal grouping by lineage prevents this.

Options considered

1. Random train/test split at record level.
2. Temporal split grouped by lineage (60/20/20).
3. Group k-fold by org.

Decision

Temporal 60/20/20 grouped by lineage (option 2), test portion locked.

Rationale

The same lineage and its corrected versions must never span splits (T14). Temporal ordering reflects deployment reality (models only see the past). Org-level k-fold was deferred; the multi-site structure is

instead exercised by the multi-site data contract (D03).

Corresponding implementation

- src/lab_billing/forecast/pipeline.py temporal_grouped_split.
- Leakage guards T10/T13/T14/T15.

Verification

- T14 (no lineage across splits), T15 (preprocessing fitted on train only), tests/test_module_b.py.
- Split sizes on the reference run: 2,419 train / 846 test interval rows (192 test snapshots).

Protocol Change PC01 — Interval granularity 1 -> 5 days

- Date: 2024-08-06
- Status: applied (protocol v1.1)
- Contributor: Applicant

Background

The first module B run (2024-08-05) at 1-day granularity produced unusable models: payment events concentrate around 10-30 days after submission, so 1-day intervals have >95% none-rows and the conditional probabilities are numerically unstable (B1 snapshot log-loss 11.3 vs B0 0.74).

Options considered

1. Keep 1-day intervals and add heavy class reweighting.
2. Change interval granularity to 5 days (H=30 becomes 6 intervals).
3. Move to a continuous-time model.

Decision

Option 2, recorded as a protocol change (not a silent edit).

Rationale

5-day intervals raise per-interval event rates into a learnable range while preserving within-horizon discrimination. Reweighting alone (option 1) distorts absolute probabilities — this was later confirmed in the balanced-class-weighting defect fixed in 2024-12. The change is versioned in research/protocol_changes.csv so the frozen protocol and

the implementation stay in sync.

Corresponding implementation

- configs/reference.json prediction.interval_days = 5.
- src/lab_billing/forecast/snapshot.py and models.py interval handling.
- research/protocol_v1.json v1.1 with change_history.

Verification

- Post-change locked-test metrics: B1 snapshot log-loss 0.669 (B0 0.736); C1 shows no incremental value (reported as-is).

Research decision records (narrative index)

These records expand research/research_decisions.csv into a

question-and-answer form: what was decided, what the alternatives were,

why this choice, where it lives in the code, and how it was verified.

Each entry traces to specific files and tests so the decision -> code ->

test chain is auditable.

- D01 2023-09-30 domain scope -> data_preparation.py SERVICE_MIX
- D02 2023-10-02 linkage philosophy -> reconstruct/rules.py, tests T04/T05
- D03 2023-10-06 site contracts -> configs/reference.json, org_profiles.csv
- D04 2023-10-09 prediction target -> forecast/snapshot.py, tests T12/T19
- D05 2023-10-12 modeling frame -> forecast/models.py, test T16
- D06 2023-10-20 evaluation split -> forecast/pipeline.py, tests T14/T15
- PC01 2024-08-06 interval granularity -> protocol v1.1, protocol_changes.csv

Exhibit 18

Version History

Changelog 0.0.1 through 1.4.0 with environment migrations and maintenance iterations.

Version history (CHANGELOG)

source: CHANGELOG.md — v1.4.0

Changelog

All notable changes to this project are documented in this file.

[1.3.0] - 2026-06-10

Changed

- Reports regenerated against the v1.3 pipeline outputs; test suite re-run after dependency refresh (Python 3.12.13, scikit-learn 1.9.1).
- Review packet rebuilt with current screenshots.

[1.2.0] - 2026-02-14

- Validation script hardened: adjacency truth derivation now tolerates non-numeric version ids; refusal ratio includes review queue.

[1.1.0] - 2025-11-10

- Personal research records organized (contribution index, decision register); external handoff notes added.
- Method report v1.1 issued with the 2025-08 validation numbers.

[1.0.0] - 2025-05-20

Added

- Validation suite v1.0 against hidden ground truth (module A precision/recall; module B ablations).
- Full-pipeline entry, Streamlit interface, Power BI adapter materials, acceptance tests T01-T25.

Research findings (reference data, locked test set)

- Module A: recall 0.99, precision 0.76; weak-evidence rule R3 accounts for most false positives; multi-candidate refusals 9.6%.
- Module B: B1 improves snapshot log-loss 0.36% over B0 (reported as a neutral result); C1 quality features add no incremental value.

[0.9.0] - 2025-03-10

- Streamlit interface v1.0; Power BI adapter (native .pbix NOT_BUILT).

[0.7.0] - 2024-12-15

- Module B v1.0: B0/B1/C1 discrete-time competing-event models; snapshot-level evaluation.

[0.5.0] - 2024-06-10

- Module A v1.0: amounts, as-of truncation, manual decisions.

[0.4.0] - 2024-03-15

- Constrained linkage rules R2/R3; transitive conflict detection.

[0.2.0] - 2024-01-25

- Module A v0.2: exact-id baseline.

[0.1.0] - 2023-11-20

- Data contract, schema validators, data preparation pipeline v0.1.

[0.0.1] - 2023-09-28

Added

- Repository and workspace skeleton initialized.
- Initial environment (Python 3.11.5, scikit-learn 1.3.0) captured in environment.json.
- Data governance templates and project state files created.
- configuration separation established (study / identified-data modes) (configs/).

E4 — Verification (Exhibits 19–24)

Exhibit 19

Validation Against Held-Out Adjudicated Labels

Module A precision/recall/refusal against held-out adjudicated adjacency;
Module B model comparison and ablations.

Validation against held-out adjudicated labels

source: outputs/runs/run-20260922T063622Z/module_b/validation_report.json (research edition)

```
{
  "module_a": {
    "truth_pairs": 95,
    "predicted_pairs": 123,
    "true_positive": 94,
    "false_positive": 29,
    "false_negative": 1,
    "precision": 0.7642,
    "recall": 0.9895,
    "refusal_ratio": 0.0956,
    "baseline_a0_unconstrained": {
      "predicted_pairs": 1164,
      "true_positive": 2,
      "false_positive": 1162,
      "false_negative": 93,
      "precision": 0.0017,
      "recall": 0.0211
    }
  },
  "module_b": {
    "snapshots": 959,
    "test_snapshots": 192,
    "outcomes": {
      "payment": 576,
      "still_open": 375,
      "close_no_payment": 8
    },
    "models": {
      "B0_age_stage_baseline": {
        "status": "FITTED",
        "snap_brier": 0.4943193812002867,
        "snap_log_loss": 0.7148682447611714,
        "n_snapshots": 192,
        "n_abstain": 0
      },
      "B1_discrete_multinomial": {
        "status": "FITTED",
        "snap_brier": 0.4908483777418546,
        "snap_log_loss": 0.7123097346939679,
        "n_snapshots": 192,
        "n_abstain": 0
      },
      "C1_quality_augmented": {
        "status": "FITTED",
        "snap_brier": 0.4914677676105672,
        "snap_log_loss": 0.7124455697623446,
        "n_snapshots": 192,
        "n_abstain": 0
      }
    }
  },
  "ablations": {
```

```
"quality_features": {
  "B1_snap_log_loss": 0.7123097346939679,
  "C1_snap_log_loss": 0.7124455697623446,
  "delta": 0.0001,
  "interpretation": "quality features add no incremental value on the locked test set"
},
"candidate_vs_baseline": {
  "B0_snap_log_loss": 0.7148682447611714,
  "B1_snap_log_loss": 0.7123097346939679,
  "relative_improvement": 0.0036
}
},
"limitations": {
  "study_scope": "All figures come from the study dataset of three participating sites.
No research claim transfers to real operations.",
  "censored_share": 0,
  "close_class_rarity": 8,
  "leakage_audit": "T10/T13/T14/T15 enforced: as-of truncation, forbidden fields,
lineage-grouped splits, train-only preprocessing."
}
}
```

Exhibit 20

Acceptance Test Report

Acceptance suites T01-T25 with observed outcomes.

Acceptance test report (T01-T25)

source: outputs/reports/test_report_20260922.md

Test Report — Acceptance Suites T01-T25

- Generated: 20260922 (from outputs/qa/junit.xml)
- Status: PASS — 35 tests, 0 failures,
0 errors, 3.0s
- Composition: 22 acceptance-suite tests (T-series), 12 regression
tests (C-series), 1 additional regression test(s)

Test reports, protocols and run receipts are distinct records: the protocol defines how each problem is verified; the receipt records program execution evidence; this report records the observed outcome. The T-series covers the acceptance suites T01–T25; C-series tests lock edge-case fixes (C-01 onward); both are executed in the same session and reported together.

Referenced run: outputs/runs/run-20260922T104226Z

Exhibit 21

Data Card

Dataset formation, tables, data-quality handling and curation rules.

Data card

source: outputs/reports/data_card_v2.0_20260922.md

Data Card — Participating-Site Billing Event Records

- Version: 2.0 (2026-09-22)

Sample formation

- 1000 billing lineages from three participating clinical laboratory organizations (site tokens CPL / TRI / NDX) with distinct ownership profiles.
- Data window 2022-07-01T00:00:00+00:00 to 2023-11-15T00:00:00+00:00; per-site local timezones with DST transitions; weekday-dominant submission and payment rhythms; a mix of same-day and lagged record entry.
- Outcome mix (adjudicated): payment ~84%, still-open ~10%, no-payment close ~6%.
- Payer mix: commercial ~55%, government ~32%, self-pay ~8%, other ~5%.

Tables

Table	Rows
bills	1167
events	2145
allocations	883
observation_windows	6

Data quality issues observed and handled

- duplicate event imports, cross-organization duplicate bill numbers, missing correction references, timezone-less timestamps, invalid amounts, non-USD currencies — isolated by the schema validator and reported; originals retained.

Curation

- Invalid rows isolated, never deleted; original records retained.
- Adjudicated reference labels (lineage membership, outcome times) are held out from all model inputs and used only for evaluation.

Limitations

- The three participating sites span three ownership models in one national context; distributions here are descriptive of these sites, not claims about

the broader industry.

- Identified patient data is outside the scope of this dataset.

Exhibit 22

Data Sample — Bills

De-identified bills table, first rows.

Data sample — bills (de-identified)

source: outputs/runs/run-20260922T104226Z/data/bills.csv — first 25 rows

(13 fields; rendered as field dictionaries to preserve all columns)

Row 1

record_id	CPL-R00001
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000001
version_id	1
previous_record_id	
payer_token	SELF
service_token	CHEM_PROFILE
billed_amount_minor	17521
currency	USD
submitted_at	2022-08-09T14:12:00-05:00
available_at	2022-08-12T23:05:29-05:00
source_ref	CPL/SRC/bills/00001

Row 2

record_id	CPL-R00002
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000002
version_id	1
previous_record_id	
payer_token	SELF
service_token	BASIC_PANEL
billed_amount_minor	13320
currency	USD
submitted_at	2023-07-11T12:53:00-05:00
available_at	2023-07-13T17:24:00-05:00
source_ref	CPL/SRC/bills/00002

Row 3

record_id	CPL-R00003
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000003

version_id	1
previous_record_id	
payer_token	MCD
service_token	CHEM_PROFILE
billed_amount_minor	15960
currency	USD
submitted_at	2023-06-16T15:26:00-05:00
available_at	2023-06-16T17:48:40-05:00
source_ref	CPL/SRC/bills/00004

Row 4

record_id	CPL-R00004
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000003
version_id	2
previous_record_id	CPL-R00003
payer_token	MCD
service_token	CHEM_PROFILE
billed_amount_minor	19217
currency	USD
submitted_at	2023-06-26T15:39:00-05:00
available_at	2023-06-28T14:51:55-05:00
source_ref	CPL/SRC/bills/00005

Row 5

record_id	CPL-R00005
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000004
version_id	1
previous_record_id	
payer_token	MCR
service_token	TOXICOLOGY
billed_amount_minor	20106
currency	USD
submitted_at	2023-07-11T08:51:00-05:00
available_at	2023-07-11T14:10:44-05:00
source_ref	CPL/SRC/bills/00007

Row 6

record_id	CPL-R00006
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000005
version_id	1
previous_record_id	
payer_token	MCD
service_token	CHEM_PROFILE
billed_amount_minor	11958
currency	USD
submitted_at	2022-07-11T10:18:00-05:00
available_at	2022-07-12T02:05:28-05:00
source_ref	CPL/SRC/bills/00009

Row 7

record_id	CPL-R00007
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000006
version_id	1
previous_record_id	
payer_token	AET
service_token	BASIC_PANEL
billed_amount_minor	7749
currency	USD
submitted_at	2023-03-07T09:27:00-06:00
available_at	2023-03-09T06:13:07-06:00
source_ref	CPL/SRC/bills/00011

Row 8

record_id	CPL-R00008
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000006
version_id	2
previous_record_id	CPL-R00007
payer_token	AET
service_token	BASIC_PANEL
billed_amount_minor	8036
currency	USD

submitted_at	2023-03-17T15:02:00-05:00
available_at	2023-03-17T16:16:30-05:00
source_ref	CPL/SRC/bills/00012

Row 9

record_id	CPL-R00009
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000007
version_id	1
previous_record_id	
payer_token	AET
service_token	BASIC_PANEL
billed_amount_minor	5411
currency	USD
submitted_at	2023-07-24T11:17:00-05:00
available_at	2023-07-27T22:19:46-05:00
source_ref	CPL/SRC/bills/00015

Row 10

record_id	CPL-R00010
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000008
version_id	1
previous_record_id	
payer_token	HUM
service_token	ANATOMIC_PATH
billed_amount_minor	14252
currency	USD
submitted_at	2023-06-19T09:57:00-05:00
available_at	2023-06-19T14:07:08-05:00
source_ref	CPL/SRC/bills/00017

Row 11

record_id	CPL-R00011
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000009
version_id	1

previous_record_id	
payer_token	BCB
service_token	CHEM_PROFILE
billed_amount_minor	22498
currency	USD
submitted_at	2022-07-11T14:20:00-05:00
available_at	2022-07-11T15:10:46-05:00
source_ref	CPL/SRC/bills/00019

Row 12

record_id	CPL-R00012
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000010
version_id	1
previous_record_id	
payer_token	HUM
service_token	BASIC_PANEL
billed_amount_minor	5507
currency	USD
submitted_at	2023-05-01T11:35:00-05:00
available_at	2023-05-12T03:00:01-05:00
source_ref	CPL/SRC/bills/00020

Row 13

record_id	CPL-R00013
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000011
version_id	1
previous_record_id	
payer_token	MCD
service_token	CHEM_PROFILE
billed_amount_minor	8419
currency	USD
submitted_at	2023-07-31T15:01:00-05:00
available_at	2023-08-03T17:07:55-05:00
source_ref	CPL/SRC/bills/00022

Row 14

record_id	CPL-R00014
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000012
version_id	1
previous_record_id	
payer_token	MCR
service_token	MICROBIOLOGY
billed_amount_minor	16370
currency	USD
submitted_at	2022-11-08T11:28:00-06:00
available_at	2022-11-12T11:15:03-06:00
source_ref	CPL/SRC/bills/00024

Row 15

record_id	CPL-R00015
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000013
version_id	1
previous_record_id	
payer_token	HUM
service_token	BASIC_PANEL
billed_amount_minor	3569
currency	USD
submitted_at	2023-02-03T13:21:00-06:00
available_at	2023-02-04T19:00:44-06:00
source_ref	CPL/SRC/bills/00026

Row 16

record_id	CPL-R00016
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000014
version_id	1
previous_record_id	
payer_token	UHC
service_token	BASIC_PANEL
billed_amount_minor	7726
currency	USD

submitted_at	2022-09-05T15:03:00-05:00
available_at	2022-09-13T02:25:42-05:00
source_ref	CPL/SRC/bills/00027

Row 17

record_id	CPL-R00017
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000015
version_id	1
previous_record_id	
payer_token	MCR
service_token	BASIC_PANEL
billed_amount_minor	13092
currency	USD
submitted_at	2022-08-18T08:24:00-05:00
available_at	2022-08-18T15:09:51-05:00
source_ref	CPL/SRC/bills/00029

Row 18

record_id	CPL-R00018
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000016
version_id	1
previous_record_id	
payer_token	UHC
service_token	CHEM_PROFILE
billed_amount_minor	14488
currency	USD
submitted_at	2022-08-31T13:13:00-05:00
available_at	2022-08-31T16:12:40-05:00
source_ref	CPL/SRC/bills/00031

Row 19

record_id	CPL-R00019
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000017
version_id	1

previous_record_id	
payer_token	SELF
service_token	BASIC_PANEL
billed_amount_minor	10456
currency	USD
submitted_at	2023-02-15T15:55:00-06:00
available_at	2023-02-21T13:38:03-06:00
source_ref	CPL/SRC/bills/00033

Row 20

record_id	CPL-R00020
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000018
version_id	1
previous_record_id	
payer_token	MCR
service_token	CHEM_PROFILE
billed_amount_minor	4457
currency	USD
submitted_at	2023-03-29T14:31:00-05:00
available_at	2023-04-02T23:09:34-05:00
source_ref	CPL/SRC/bills/00035

Row 21

record_id	CPL-R00021
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000019
version_id	1
previous_record_id	
payer_token	MCD
service_token	CHEM_PROFILE
billed_amount_minor	9667
currency	USD
submitted_at	2022-10-03T14:43:00-05:00
available_at	2022-10-07T07:45:01-05:00
source_ref	CPL/SRC/bills/00036

Row 22

record_id	CPL-R00022
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000020
version_id	1
previous_record_id	
payer_token	CIG
service_token	CHEM_PROFILE
billed_amount_minor	11816
currency	USD
submitted_at	2022-11-30T09:43:00-06:00
available_at	2022-12-01T20:06:56-06:00
source_ref	CPL/SRC/bills/00037

Row 23

record_id	CPL-R00023
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000021
version_id	1
previous_record_id	
payer_token	MCR
service_token	BASIC_PANEL
billed_amount_minor	9072
currency	USD
submitted_at	2023-06-05T15:32:00-05:00
available_at	2023-06-08T19:14:52-05:00
source_ref	CPL/SRC/bills/00039

Row 24

record_id	CPL-R00024
org_token	CPL
source_system	CPL_BILLING
bill_id	2022-4000022
version_id	1
previous_record_id	
payer_token	UHC
service_token	BASIC_PANEL
billed_amount_minor	10731
currency	USD

submitted_at	2023-08-14T15:36:00-05:00
available_at	2023-08-15T10:35:35-05:00
source_ref	CPL/SRC/bills/00041

Row 25

record_id	CPL-R00025
org_token	CPL
source_system	CPL_BILLING
bill_id	2023-4000023
version_id	1
previous_record_id	
payer_token	UHC
service_token	MOLECULAR
billed_amount_minor	34932
currency	USD
submitted_at	2022-07-07T10:05:00-05:00
available_at	2022-07-08T06:51:55-05:00
source_ref	CPL/SRC/bills/00043

(first 25 of 1167 rows shown)

Exhibit 23

Data Sample — Events

De-identified events table, first rows.

Data sample — events (de-identified)

source: outputs/runs/run-20260922T104226Z/data/events.csv — first 25 rows

(12 fields; rendered as field dictionaries to preserve all columns)

Row 1

event_id	EV000001
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00001
event_type	SUBMITTED
event_at	2022-08-09T14:12:00-05:00
available_at	2022-08-09T17:51:28-05:00
amount_minor	17521
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00001

Row 2

event_id	EV000002
org_token	CPL
source_system	CPL_BILLING
bill_record_id	
event_type	PAYMENT_POSTED
event_at	2022-08-18T08:24:00-05:00
available_at	2022-08-18T10:17:42-05:00
amount_minor	17521
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00002

Row 3

event_id	EV000003
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00002
event_type	SUBMITTED
event_at	2023-07-11T12:53:00

available_at	2023-07-11T14:12:07
amount_minor	13320
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00003

Row 4

event_id	EV000004
org_token	CPL
source_system	CPL_BILLING
bill_record_id	
event_type	PAYMENT_POSTED
event_at	2023-07-18T11:19:00-05:00
available_at	2023-07-18T15:07:34-05:00
amount_minor	13320
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00004

Row 5

event_id	EV000005
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00003
event_type	SUBMITTED
event_at	2023-06-16T15:26:00-05:00
available_at	2023-06-20T07:36:19-05:00
amount_minor	15960
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00005

Row 6

event_id	EV000006
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00004

event_type	CORRECTED
event_at	2023-06-26T15:39:00-05:00
available_at	2023-06-26T16:39:17-05:00
amount_minor	19217
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00006

Row 7

event_id	EV000007
org_token	CPL
source_system	CPL_BILLING
bill_record_id	
event_type	PAYMENT_POSTED
event_at	2023-06-28T08:45:00-05:00
available_at	2023-07-03T10:56:57-05:00
amount_minor	19217
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00007

Row 8

event_id	EV000008
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00005
event_type	SUBMITTED
event_at	2023-07-11T08:51:00-05:00
available_at	2023-07-14T20:00:27-05:00
amount_minor	20106
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00008

Row 9

event_id	EV000009
org_token	CPL

source_system	CPL_BILLING
bill_record_id	
event_type	PAYMENT_POSTED
event_at	2023-07-20T10:20:00-05:00
available_at	2023-07-20T16:08:23-05:00
amount_minor	20106
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00009

Row 10

event_id	EV000010
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00006
event_type	SUBMITTED
event_at	2022-07-11T10:18:00-05:00
available_at	2022-07-11T17:59:56-05:00
amount_minor	11958
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00010

Row 11

event_id	EV000011
org_token	CPL
source_system	CPL_BILLING
bill_record_id	
event_type	PAYMENT_POSTED
event_at	2022-08-08T14:58:00-05:00
available_at	2022-08-14T10:59:28-05:00
amount_minor	11958
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00011

Row 12

event_id	EV000012
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00007
event_type	SUBMITTED
event_at	2023-03-07T09:27:00-06:00
available_at	2023-03-20T02:14:58-05:00
amount_minor	7749
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00012

Row 13

event_id	EV000013
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00008
event_type	CORRECTED
event_at	2023-03-17T15:02:00-05:00
available_at	2023-03-17T19:46:32-05:00
amount_minor	8036
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00013

Row 14

event_id	EV000014
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00007
event_type	VOID_VERSION
event_at	2023-03-17T15:02:00-05:00
available_at	2023-03-19T17:47:57-05:00
amount_minor	7749
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00014

Row 15

event_id	EV000015
org_token	CPL
source_system	CPL_BILLING
bill_record_id	
event_type	PAYMENT_POSTED
event_at	2023-03-27T09:57:00-05:00
available_at	2023-03-31T09:33:14-05:00
amount_minor	8036
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00015

Row 16

event_id	EV000016
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00009
event_type	SUBMITTED
event_at	2023-07-24T11:17:00-05:00
available_at	2023-07-25T03:15:19-05:00
amount_minor	5411
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00016

Row 17

event_id	EV000017
org_token	CPL
source_system	CPL_BILLING
bill_record_id	
event_type	PAYMENT_POSTED
event_at	2023-10-02T08:56:00-05:00
available_at	2023-10-02T16:16:11-05:00
amount_minor	5411
currency	USD
reverses_event_id	
terminal_flag	false

source_ref	CPL/SRC/events/00017
------------	----------------------

Row 18

event_id	EV000018
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00010
event_type	SUBMITTED
event_at	2023-06-19T09:57:00-05:00
available_at	2023-06-19T17:26:41-05:00
amount_minor	14252
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00018

Row 19

event_id	EV000019
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00010
event_type	CLOSED_NO_PAYMENT
event_at	2023-08-28T16:59:00-05:00
available_at	2023-09-10T03:37:20-05:00
amount_minor	
currency	
reverses_event_id	
terminal_flag	true
source_ref	CPL/SRC/events/00019

Row 20

event_id	EV000020
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00011
event_type	SUBMITTED
event_at	2022-07-11T14:20:00-05:00
available_at	2022-07-15T03:21:35-05:00
amount_minor	22498
currency	USD

reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00020

Row 21

event_id	EV000021
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00012
event_type	SUBMITTED
event_at	2023-05-01T11:35:00-05:00
available_at	2023-05-03T14:07:50-05:00
amount_minor	5507
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00021

Row 22

event_id	EV000022
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00012
event_type	CLOSED_NO_PAYMENT
event_at	2023-06-20T10:07:00-05:00
available_at	2023-06-20T21:30:30-05:00
amount_minor	
currency	
reverses_event_id	
terminal_flag	true
source_ref	CPL/SRC/events/00022

Row 23

event_id	EV000023
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00013
event_type	SUBMITTED
event_at	2023-07-31T15:01:00-05:00
available_at	2023-08-02T13:38:57-05:00

amount_minor	8419
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00023

Row 24

event_id	EV000024
org_token	CPL
source_system	CPL_BILLING
bill_record_id	
event_type	PAYMENT_POSTED
event_at	2023-10-23T14:31:00-05:00
available_at	2023-10-26T01:27:36-05:00
amount_minor	8419
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00024

Row 25

event_id	EV000025
org_token	CPL
source_system	CPL_BILLING
bill_record_id	CPL-R00014
event_type	SUBMITTED
event_at	2022-11-08T11:28:00-06:00
available_at	2022-11-10T03:29:03-06:00
amount_minor	16370
currency	USD
reverses_event_id	
terminal_flag	false
source_ref	CPL/SRC/events/00025

(first 25 of 2145 rows shown)

Exhibit 24

Data Sample — Allocations

De-identified allocations table, first rows.

Data sample — allocations (de-identified)

source: outputs/runs/run-20260922T104226Z/data/allocations.csv — first 25 rows

allocation_id	payment_event_id	bill_record_id	allocated_amount_minor	currency	available_at	source_ref
AL000001	EV000002	CPL-R00001	17521	USD	2022-08-18T15:02:17-05:00	CPL/SRC/allocations/00002
AL000002	EV000004	CPL-R00002	13320	USD	2023-07-18T15:10:29-05:00	CPL/SRC/allocations/00004
AL000003	EV000007	CPL-R00004	19217	USD	2023-06-28T12:18:00-05:00	CPL/SRC/allocations/00007
AL000004	EV000009	CPL-R00005	20106	USD	2023-07-20T14:23:29-05:00	CPL/SRC/allocations/00009
AL000005	EV000011	CPL-R00006	11958	USD	2022-08-08T22:11:50-05:00	CPL/SRC/allocations/00011
AL000006	EV000015	CPL-R00008	8036	USD	2023-03-31T18:38:31-05:00	CPL/SRC/allocations/00015
AL000007	EV000017	CPL-R00009	5411	USD	2023-10-02T13:25:00-05:00	CPL/SRC/allocations/00017
AL000008	EV000024	CPL-R00013	5004	USD	2023-10-23T17:55:39-05:00	CPL/SRC/allocations/00024
AL000009	EV000026	CPL-R00014	12124	USD	2022-11-18T11:20:10-06:00	CPL/SRC/allocations/00026
AL000010	EV000029	CPL-R00016	7726	USD	2022-10-03T12:39:20-05:00	CPL/SRC/allocations/00029
AL000011	EV000031	CPL-R00017	13092	USD	2022-08-29T21:53:09-05:00	CPL/SRC/allocations/00031
AL000012	EV000033	CPL-R00018	11939	USD	2022-10-21T22:06:35-05:00	CPL/SRC/allocations/00033
AL000013	EV000035	CPL-R00019	10456	USD	2023-02-24T00:48:37-06:00	CPL/SRC/allocations/00035
AL000014	EV000039	CPL-R00022	6559	USD	2022-12-30T03:53:52-06:00	CPL/SRC/allocations/00039

allocation_id	payment_event_id	bill_record_id	allocated_amount_minor	currency	available_at	source_ref
AL000015	EV000041	CPL-R00023	9072	USD	2023-07-04T15:33:12-05:00	CPL/SRC/allocations/00041
AL000016	EV000043	CPL-R00024	10731	USD	2023-10-06T19:37:01-05:00	CPL/SRC/allocations/00043
AL000017	EV000045	CPL-R00025	27610	USD	2022-07-21T22:20:24-05:00	CPL/SRC/allocations/00045
AL000018	EV000050	CPL-R00028	37472	USD	2023-03-16T01:07:48-05:00	CPL/SRC/allocations/00050
AL000019	EV000054	CPL-R00031	59848	USD	2023-08-01T17:05:58-05:00	CPL/SRC/allocations/00054
AL000020	EV000056	CPL-R00032	5056	USD	2023-07-26T08:31:03-05:00	CPL/SRC/allocations/00056
AL000021	EV000058	CPL-R00033	18704	USD	2023-04-06T14:53:46-05:00	CPL/SRC/allocations/00058
AL000022	EV000060	CPL-R00034	36864	USD	2023-09-14T05:51:17-05:00	CPL/SRC/allocations/00060
AL000023	EV000064	CPL-R00036	13410	USD	2022-12-13T18:58:11-06:00	CPL/SRC/allocations/00065
AL000024	EV000064	CPL-R00037	8294	USD	2022-12-12T22:16:09-06:00	CPL/SRC/allocations/00065
AL000025	EV000068	CPL-R00039	2848	USD	2023-09-21T14:31:51-05:00	CPL/SRC/allocations/00068

(first 25 of 883 rows shown)